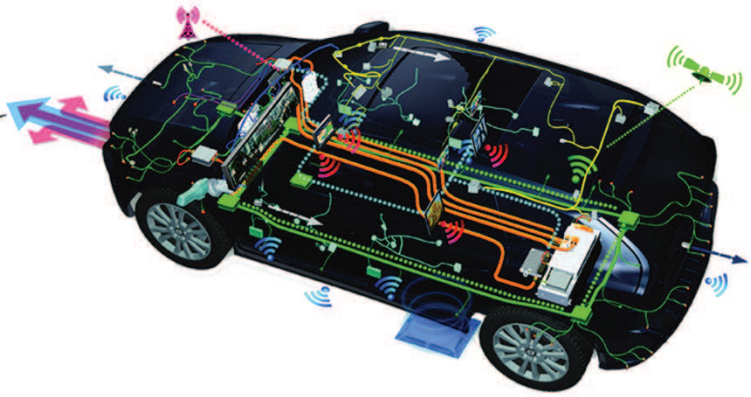
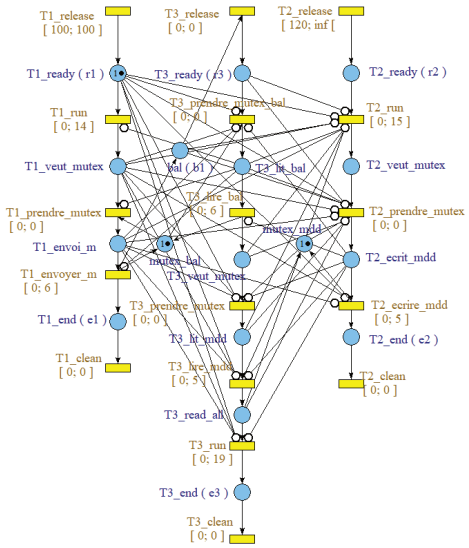
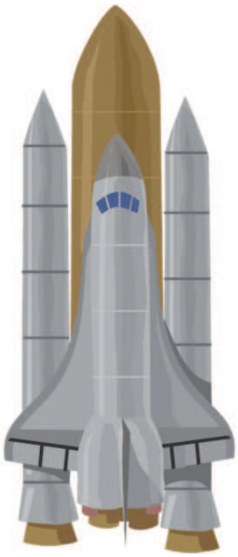
Ecole Nationale Supérieure
de Mécanique et d'Aérotechnique
Téléport 2, 1 avenue Clément Ader - B.P. 40109 - 86961 FUTUROSCOPE Cedex
(FRANCE)



Validation Temporelle
Response-Time Analysis, Model Checking





Emmanuel Grolleau
grolleau@ensma.fr
Bureau B414

Janvier 2025

TD VAT flottant : Complexité algorithmique

La complexité représente la substantifique moëlle d'un problème. Chaque problème possède sa propre complexité intrinsèque.

1 Complexité d'un algorithme vs. d'un problème

Soit T un tableau d'entiers de taille n, **non trié**. Voici un algorithme testant si le tableau possède une valeur E donnée.

```
Inclue (T : tableau de n entiers, E : entier)
-- Renvoie vrai ssi E<max(T)
Pour i de 1 à n
      Si T(i)=E retourner vrai
Retourner faux
```

Ici, ce qui peut varier est n, la taille du tableau : 10 ou 1 milliard ? Les valeurs numériques sont assumées codées sur des entiers logeant dans des registres, 32 ou 64 bits par exemple. Par conséquent, nous supposons que les opérations de comparaison (instruction dans la boucle pour, ou même traitement de l'indice de boucle) ont des durées constantes. La taille de l'entrée est (2+n)*taille entier, taille du tableau, entier E, et tableau lui-même. Tout étant constant à part n, nous disons donc que l'entrée a une taille n donnée par la taille du tableau.

a. Quelle est la complexité de cet algorithme en fonction de n ?

Maintenant observons le problème, peut-on faire mieux ? Par exemple, la mémoire cache effectue cette recherche de clé à chaque requête à la RAM. Pensez-vous qu'elle parcoure les valeurs de clés stockées en cache une par une ? Non, une mémoire associative permet, da façon matérielle, une recherche en parallèle sur toutes les clés. Une mémoire associative n'est pas une architecture Von Neumann. Il est complexe de modéliser un ordinateur complet, Alan Turing a proposé une abstraction très simple, que l'on nomme **Machine de Turing** :

Une machine de Turing est définie par une tête de lecture/écriture sur un ruban infini à gauche et à droite de la tête. Le ruban est constitué de cases pouvant contenir des symboles de l'alphabet ou le symbole blanc (noté b ou ε), qui remplit toutes les cases non initialisées du ruban. Ce ruban correspond à une mémoire séquentielle. La tête suit un algorithme qui est en fait une machine à états, qui dépend du symbole lu, et de l'état courant. En fonction de cela, la transition donne le symbole à écrire, et l'action à effectuer : un pas à gauche L, ou un pas à droite R. Un état spécial, Halt, est atteint lorsque c'est un état final de la machine à états, ou bien si aucune transition de l'état courant avec le symbole lu n'existe (cela correspondrait à un plantage). On peut par exemple imaginer un comptage en base 1, comme sur cet exemple, tiré d'un TD du Dr. Steven J. Zeil.

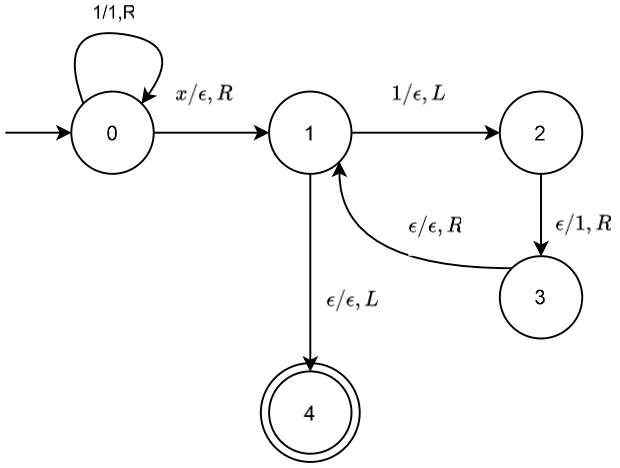


Figure 1 : Système d'instructions d'une machine de Turing additionnant deux nombres en base 1. L'entrée est donnée sur le ruban à droite de la tête, sous forme 111x11 par exemple pour 3+2, ici nous devons obtenir 11111.

b. Vérifier que pour l'entrée 111x11, avec la tête sur le premier 1, on obtient bien 11111 avec la tête sur le dernier 1.
c. Qu'ajouter pour que la tête soit positionnée sur le premier 1 ?
d. Que se passe-t-il si on a un symbole non reconnu en entrée ?
e. Supposons que nous voulions faire un programme Incrémenter, mais d'un entier donné en décimale. La tête est initialement placée sur le chiffre de poids faible, à droite du ruban. Le programme doit incrémenter l'entier et placer à la fin la tête sur le chiffre de poids faible. Donner le programme sous forme d'un automate comme sur la Figure 1.

La Machine de Turing ne va pas nous servir à calculer une complexité, car l'absence d'un accès direct à la mémoire (une bande est une mémoire séquentielle) va augmenter le nombre d'opérations nécessaires. Cependant, elle est équivalente en puissance d'expression à une Random Access Machine qui représente registres, RAM et opérations de base d'un microprocesseur. C'est sur ce type de modèle que la complexité d'un programme est assumée.

La Machine de Turing a notamment été utilisée pour étudier la décidabilité. Un problème est décidable si une réponse au problème peut être donnée en temps fini. Elle sert à démontrer par l'absurde que le problème de l'arrêt d'un programme est indécidable.

f. Supposons maintenant que le tableau d'entrée est trié par ordre croissant. Quelle est la complexité de recherche d'un élément ?
g. Notez que la fonction Inclue fonctionne sur un tableau trié, mais que dire de la complexité de cet algorithme par rapport à la complexité du problème ?

Visiter le site https://www.bigocheatsheet.com/ qui met en évidence des problèmes et complexités d'opérations classiques.

2 Classes de complexité des problèmes

Comment savoir si un problème est intrinsèquement complexe, par exemple s'il ne permet pas de solution polynomiale sur une architecture Von Neumann ?

Classification par classes de complexité P (polynomial), NP (non-déterministe polynomial), co-NP (dont le complémentaire est NP) ou NP^{coNP}, $CoNP^{NP}$, $NP^{CoNP\wedge NP}$ etc.

Considérons les problèmes de décision, c'est-à-dire les algorithmes qui répondent OUI/NON, comme c'était le cas pour l'algorithme Inclue. Notons qu'un problème de recherche se transforme en problème de décision, ainsi trouver le plus court chemin dans un graphe, se transforme en existe-t-il un chemin de longueur <= k ? Nous voulons classifier ces problèmes suivant leur complexité.

Problème de référence : SATISFIABILITY (SAT), et même 3SAT sa version limitée à la satisfiabilité de formule booléenne constituées de conjonctions de disjonctions de 3 littéraux.

Problème **3SAT** : soit une formule booléenne constituée d'une conjonction de disjonctions de 3 littéraux (exemple : (a ou non b ou c) et (non a ou non b ou c)). Existe-t-il une valuation des variables booléennes telle que la formule est vraie ?

Réponse : il nous faut pour le problème général tester toutes les valuations possibles.

a. Utiliser un arbre représentant toutes les valuations possibles pour (a ou non b ou c) et (non a ou non b ou c). Au premier niveau on représente a, qui vaut vrai ou faux, b au second, et c au troisième niveau, toutes les valeurs possibles étant représentées au niveau de chaque feuille.
b. Quelle est la taille d'un arbre d'une formule constituée de n variables ?
c. Si la réponse est OUI, quelle est la complexité de la vérification de cette réponse ?
d. Si la réponse est NON, quelle est la complexité de la vérification de cette réponse ?

3SAT est l'un des problèmes **NP-complets**. Il en existe des centaines, recensés notamment dans [1]. Un problème est NP si exhiber un exemple permet de démontrer que la réponse est OUI, alors que démontrer que la réponse est NON requière l'arbre entier.

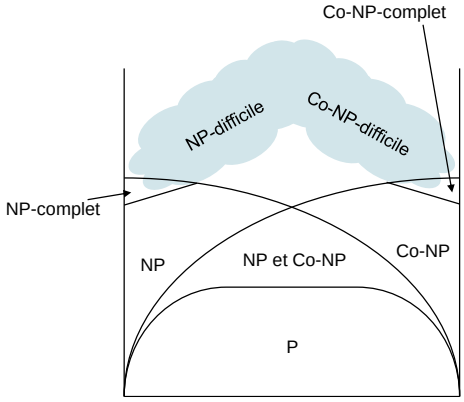


Figure 2 : une représentation possible des classes de complexité P, NP et Co-NP

Les problèmes **Co-NP** sont complémentaires dans le sens où dire NON requière d'exhiber un contre-exemple, alors que dire OUI requière l'arbre entier.

e. Le problème CONTRADICTION qui renvoie OUI ssi la formule donnée est une contradiction est-il NP ou Co-NP ?
f. Qu'en est-il du problème TAUTOLOGIE ?

La conjecture usuellement assumée est que P≠NP, c'est-à-dire qu'il n'existe pas d'algorithme polynomial (sur architecture Von Neumann) permettant de résoudre des problèmes NP. La preuve de cette conjecture fait partie des 7 problèmes du millénaire.

3 Réduction polynomiale

Etant donné un nouveau problème, pour lequel nous décidons de proposer un algorithme optimal en temps exponentiel, ou une heuristique polynomiale, comment être sûr que l'on ne pourrait pas proposer un algorithme optimal polynomial ?

En démontrant qu'il est aussi dur qu'un problème NP-complet, nous montrons qu'il est **NP-difficile**. Si on souhaite montrer qu'il NP-complet, il faut juste montrer qu'il est NP (vérification réponse donnée en temps polynomial).

Soit le problème d'ordonnancement classique PD2|res1..|Cmax qui signifie minimiser le ***makespan*** de l'exécution de jobs sur deux processeurs dédiés, sachant que les jobs partagent une ressource. Ce problème arrive lorsqu'on rince les circuits hydrauliques d'un avion civile. Noter que ce problème n'est pas un problème de décision, sa version décisionnelle serait PD2|res1..|Cmax≤k pour un k donné.

a. Proposer une stratégie polynomiale résolvant de façon optimale PD2|res1..|Cmax

Le souci est que lorsqu'on rince des circuits hydrauliques, il y a des contraintes de précédences entre jobs de façon à ne pas re-salir un tuyau en en rinçant un autre. Le problème devient alors PD2|res1..,localprecedence|Cmax≤k qui n'avait jamais été étudié dans la littérature. Notre stratégie ne fonctionne pas, si les précédences ne sont pas consistantes avec elle… Existe-t-il une stratégie optimale polynomiale ?

Considérons le problème NP-complet 3-PARTITION : soit un ensemble de 3n entiers positifs, dont la somme totale est nB. Est-il possible de les regrouper en n triplets dont la somme individuelle est B ?

Essayons de montrer que PD2|res1..,localprecedence|Cmax≤k est aussi dur que 3PART en proposant un moyen (qui doit s'exécuter en temps polynomial) de transformer toute instance de problème 3PART en une instance de notre problème, telle que si on pouvait résoudre notre problème en temps polynomial, alors on serait capable de résoudre 3PART en temps polynomial. Cela s'appelle une **réduction polynomiale**.

Soit un problème 3SAT donné sous forme $\{i_1, i_2, etc, i_{3n}\}$ avec la somme qui vaut nB. Créons une instance de PD2|res1..|Cmax≤k de la façon suivante :

- Nous supposons deux machines, partageant une ressource r.
- Sur machine 1, nous affectons 3n jobs, de durée respective i_1, i_2, etc, i_{3n} qui utilisent tous la ressource R.
- Sur machine 2, nous affectons B jobs de durée n n'utilisant pas la ressource, et B jobs de durée 1 utilisant la ressource. On introduit des contraintes de précédence entre ces jobs pour forcer les jobs de durée 1 à s'intercaler entre les jobs de durée n.

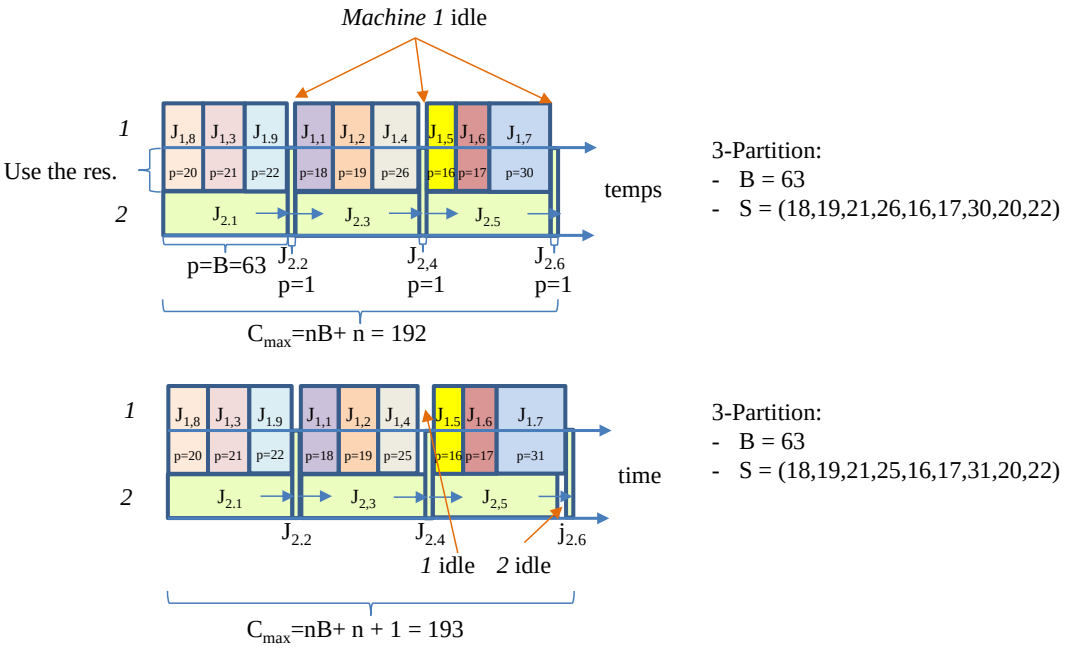


Figure 3 : Modélisation de deux problèmes 3PART sous forme de problèmes PD2|res1..|Cmax

b. Observez la Figure 3 et en déduire :

- Quel est le makespan minimal atteignable ?
- Dans quel cas peut-on l'atteindre ?
- Que peut-on en déduire sur la complexité de PD2|res1..|Cmax≤k?
- Ce problème est-il NP ?
- Que peut-on en déduire sur sa complexité ?
- Que peut-on en déduire de la complexité de PD2|res1..|Cmax ?

4 Sens fort sens faible

Dans la littérature du temps réel, on distingue deux classes de problèmes NP : sens fort et sens faible. Les problèmes NP-difficiles au sens faible sont ceux pour lesquels si l'entrée était donnée en base 1, ils seraient P. On parle de problèmes que l'on peut résoudre en temps pseudo-polynomial.

Par exemple, le crible d'Eratosthène qui permet de décomposer un nombre en facteurs premiers est pseudo-polynomial. Le calcul de temps de réponse de tâches indépendantes ordonnancées par un algorithme à priorités fixes est NP-difficile au sens faible. Vérifier le respect d'échéances d'un système de tâches sous contraintes de ressources partagées est NP-difficile au sens fort.

5 Bibliographie

[1] M. R. Garey et D. S. Johnson, *Computers and intractability: a guide to the theory of NP-completeness*. in A series of books in the mathematical sciences. New York: W. H. Freeman, 1979.

TD VAT 1 : Analyse de pire durée d'exécution

La durée d'exécution d'un programme dépend de plusieurs facteurs :

- les instructions élémentaires le composant ;
- le processeur et sa vitesse. Naïvement, on pourrait penser que chaque instruction prend 1 cycle, et donc, fréquence 2x plus élevée donne traitement 2x plus rapide. Ceci s'approche de la réalité sur des microcontrôleurs ancienne génération, mais est de plus en plus faux. C'est en tout cas totalement faux sur un microprocesseur ;
- l'état interne du processeur, qui peut par exemple contenir des pipelines ;
- l'état de la mémoire cache : l'instruction à exécuter, et les données requises, sont-elles en cache (accès 1 à 2 cycles) ou dans la RAM (accès en 3 à 10 cycles) ou stockées sur le disque dur (swap) car nous utilisons de la mémoire virtuelle (accès en plusieurs dizaines de milliers de cycles)

Il existe deux familles de méthodes pour analyser le WCET (Worst-Case Exécution Time) d'un programme sur un calculateur.

- Analyse statique de WCET, utilise un programme qui doit avoir un modèle précis du calculateur employé. Exemples : OTAWA, aiT, RapiTime, Heptane, etc. Sûre (jamais optimiste) mais nécessite un modèle précis de l'architecture d'exécution. Pessimiste lorsqu'il y a des caches et pipelines car toute inconnue sur l'état du système est modélisée de façon conservative.
- Analyse dynamique de WCET (voir Figure 1), qui consiste à instrumenter le code avec des mesures d'horloge (logicielles ou matérielles – horloges dans le CPU, analyseur logique), et mesurer de nombreuses exécutions. La difficulté est qu'il est généralement impossible de mesurer tout état interne et tout chemin (couverture du code). La solution est donc non sûre.
    - Solutions mixtes avec considération de la couverture ;
    - WCET probabiliste – distribution cumulative (cdf) de pourcentage des exécution vs. pire durée.

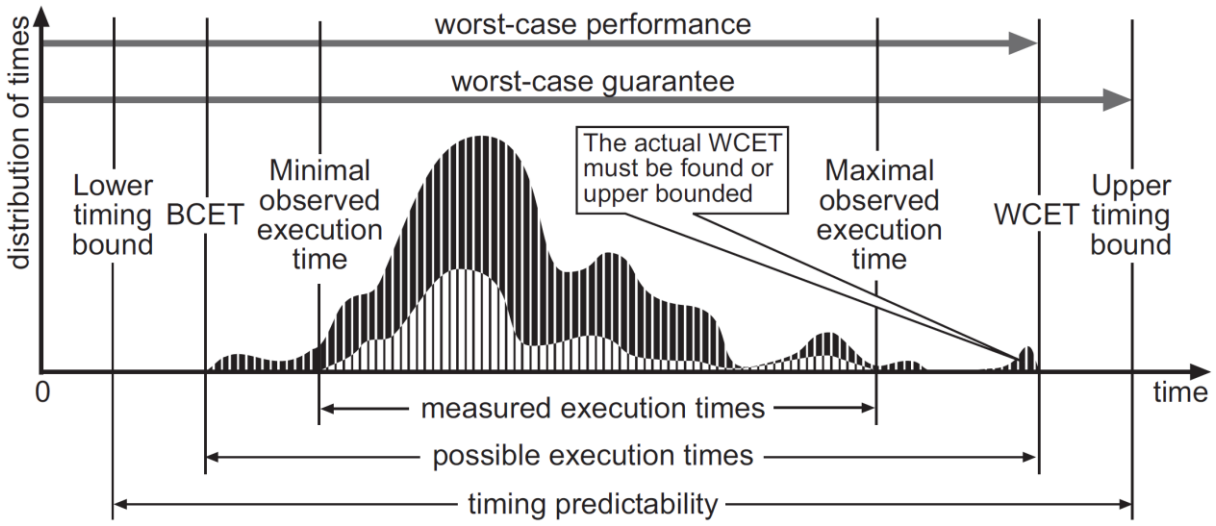


Figure 1 : Distribution typique d'une durée d'exécution, selon [1], l'un des survey les plus cités dans le domaine de la Timing Analysis (analyse de WCET)

1 Analyse statique avec OTAWA

Note : la partie du TD portant sur OTAWA a été conçue par Antoine Bertout (LIAS).

Nous allons tester OTAWA qui est un framework d'analyse statique développé à l'IRIT (Institut de Recherche en Informatique de Toulouse). Dans son principe, OTAWA décode le binaire du programme, construit son CFG (graphe de flot de contrôle) et abstrait finement le matériel. Il combine ces informations pour borner le nombre de fois que les différentes parties du programme (blocs de bases) peuvent être exécutés et le temps (nb cycles) d'exécution de chacune de ces parties. Nous ferons l'hypothèse que ce programme est destiné à être exécuté sur un microcontrôleur ARM LPC2138 de la compagnie NXP car le modèle de cette architecture est déjà disponible dans OTAWA.

1.1 Préparation

Sous Linux :

```
xhost +local:

docker run -it --rm -e DISPLAY=:0.0 --net=host registry-
ia.ensma.fr/svtr /bin/bash
```

Sous Windows avec WSL2 :

```
docker run -it --rm -v /tmp/.X11-unix:/tmp/.X11-unix -e DISPLAY -e
WAYLAND_DISPLAY -e XDG_RUNTIME_DIR -e PULSE_SERVER --net=host
registry-ia.ensma.fr/svtr /bin/bash
```

1.2 Utilisation d'OTAWA

Plusieurs programmes C sont disponibles dans le répertoire wcet, comme bs.c avec une simple fonction main qui appelle la fonction de recherche dichotomique binary_search dans un tableau.

1. Commencer par visualiser le programme.

```
cd wcet ; cat bs.c
```

2. Compiler le programme avec gcc pour architecture ARM (on parle de compilation croisée)

```
arm-linux-gnueabi-gcc -g3 -fno-stack-protector -marm -march=armv4 -o
bs.elf bs.c -nostartfiles -static -T link-flash.ld;
```

3. Lancer l'estimation de WCET de la fonction main avec OTAWA.

```
owcet -s lpc2138 bs.elf
```

En essayant d'interpréter la sortie de owcet, on peut comprendre qu' OTAWA a besoin de connaître le nombre maximal de fois que la boucle de la fonction binary_search peut être appelée (borne de boucle). Quelle est cette borne N (pour rappel, le tableau possède 15 éléments) ?

4. Nous allons préciser cette borne manuellement en éditant (avec vim) un fichier ff (flowfact) produit par la commande suivante.

```
mkff bs.elf > bs.ff
```

Dans bs.ff remplacer le symbole ? par la valeur de N trouvée précedemment. Dans vim, taper i pour passer en mode édition, échap pour sortir du mode édition, et :wq pour sauvegarder et quitter vim.

5. Relancer owcet en liant le fichier de flow fact et obtenir le WCET du programme sur le microcontrôleur.

```
owcet -s lpc2138 bs.elf -f bs.ff
```

6. Fort heureusement, l'outil orange d'OTAWA permet de déterminer automatiquement cette borne de boucle, vérifiez la valeur trouvée dans le fichier ffx en comparant avec la vôtre.

```
orange bs.c main -o bs.ffx
```

7. Nous allons générer un ensemble de statistiques qui vont nous permettre de visualiser le CFG du programme annoté.

```
owcet -s lpc2138 bs.elf --stats -S
```

8. Lancer l'interface graphique de visualisation du CFG.

```
obviews.py bs main
```

9. Recompilez le programme avec l'option -O1, relancez la commande owcet du 7. Et observez les différences de WCET.

2 Analyse dynamique

Télécharger bs.c sur votre machine, à partir du dossier Resources du TD sur Moodle. La fonction de recherche dichotomique est la même que celle que nous avons mesurée statiquement pour la cible LPC2138. Supposons que le programme doive s'exécuter sur le CPU de votre machine. Il est probable que le modèle de votre CPU ne soit pas disponible pour un analyseur de WCET statique. Nous allons donc effectuer des mesures dynamiques.

1. Compiler et lancer plusieurs fois le programme. Comment peut-on expliquer les variations observées ?

2. Modifier la valeur d'indice de fin de boucle for, en la passant à 10, 100, 1000. Comment expliquez-vous la variation décroissante de la moyenne observée de chaque durée d'exécution ?

3 Bibliographie

[1] R. Wilhelm *et al.*, « The worst-case execution-time problem—overview of methods and survey of tools », *ACM Trans. Embed. Comput. Syst.*, vol. 7, n° 3, p. 36:1-36:53, mai 2008, doi: 10.1145/1347375.1347389.

TD VAT 2
Validation par simulation et viabilité

La simulation consiste à tracer le fonctionnement attendu de l'ordonnancement d'un système. Elle ne permet de représenter qu'un comportement unique : pour une période donnée pour chaque tâche, une durée donnée, une date de réveil donnée. Cependant, nous avons vu que la durée d'exécution en-ligne était variable, qu'il existait des taches sporadiques dont la période représente en fait le délai minimal entre deux activations successives, et que certains systèmes sont concrets (dates de réveil connues) alors que d'autres sont non-concrets (dates de réveil inconnues).

La simulation permet de valider un système si et seulement si elle représente le comportement pire cas du système. Cela n'est vrai que si la simulation est **viable** dans le **contexte** pour les paramètres qui ne sont pas fixés.

Contexte : hypothèses s'appliquant au système étudié. Par exemple : « ordonnanceur en-ligne sur un système monoprocesseur, tâches périodiques simultanées indépendantes, et à échéances sur requête. »
Quelques éléments liés au contexte :
Processeur : monoprocesseur, multiprocesseur global, multiprocesseur partitionné, réparti
Ordonnanceur : en-ligne ou hors-ligne
Taches : concrètes ou non
Tâches concrètes : simultanées ou non
Activations des tâches : périodiques, sporadiques, apériodiques, reliées
Interdépendances des tâches : indépendantes, avec sections critiques (ou portions non préemptibles), contraintes de précédence
Structure interne des tâches : avec ou sans suspension

Viabilité :
Un test d'ordonnançabilité est **C-viable** dans un contexte si réduire la durée d'exécution ne peut augmenter de temps de réponse.
Il est **T-viable** dans un contexte si augmenter une période ne peut augmenter un temps de réponse.
Il est **r-viable** si augmenter une date de réveil ne peut augmenter un temps de réponse.

Exercice 1 :

Soit un système de tâches indépendantes, périodiques, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	**WCET C_i**	**Délai critique D_i**	**Période T_i**	**Priorité Π_i**
τ_1	0	8	30	30	1
τ_2	0	6	20	20	2
τ_3	0	4	15	15	3

L'**utilisation processeur** $U = \sum_{i=1}^{n} \frac{C_i}{T_i}$ (aussi appelée charge processeur) d'un système fournit une condition nécessaire triviale d'ordonnançabilité. Il faut en effet que U≤k pour qu'un système ait une

chance d'être ordonnançable sur un système de k processeurs. Si U>1, le système est trivialement non-ordonnançable sur 1 processeur.

a) Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
b) Tracer la simulation de l'ordonnancement du système, puis vérifier que la séquence produite est la même que celle produite par SimSo Web (http://projects.laas.fr/simso/simso-web/#/configuration).
c) Quel est le pire temps de réponse observé de chaque tâche ? Chaque tâche respecte-t-elle toutes ses échéances ?

Lorsque les tâches sont indépendantes, préemptibes, et ordonnancées sur un processeur, la **simulation est C-viable et T-viable**.
De plus, si le système est non concret, la simulation du système de tâches en considérant des réveils simultanés est **r-viable**.

d) Le système est-il ordonnançable par Rate Monotonic ?
e) En supposant que les dates de réveil soient inconnues, le système est-il ordonnançable ?
f) Tracer la simulation de l'ordonnancement du système en supposant cette fois un ordonnanceur Earliest Deadline First, puis vérifier que la séquence produite est la même que celle produite par SimSo Web.
g) Quel est le pire temps de réponse de chaque tâche ?
h) Le système est-il ordonnançable par EDF ?

Exercice 2 :

Considérons un système de tâches différées, dont les paramètres sont donnés ci-après :

	Date de réveil r_i	**WCET C_i**	**Délai critique D_i**	**Période T_i**	**Priorité Π_i**
τ_1	8	8	30	30	1
τ_2	4	6	20	20	2
τ_3	0	6	15	15	3

a) Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.

Si le système est ordonnançable, en monoprocesseur, avec $D_i \leq T_i$, la première fenêtre de taille H=PPCM(T_i) contenant exactement H(1-U) temps creux donne le cycle de l'ordonnancement. Ceci arrive au plus tard à la date max(r_i)+2H.

b) Tracer la simulation de l'ordonnancement du système, puis vérifier que la séquence produite est la même que celle produite par SimSo Web.
c) Tracer la simulation en supposant que les tâches sont simultanées, et conclure.

Pour une tâche de priorité Π_i, le cas le plus défavorable survient lorsqu'elle subit la pire interférence de toutes les tâches de priorité plus élevée. L'**instant critique** d'une tâche survient lorsqu'elle est activée simultanément avec toutes les tâches plus prioritaires, car cela maximise l'interférence de celles-ci.

d) En supposant les dates de réveil données dans la table, mais à supposer cette fois que les tâches sont sporadiques, que conclure ?

Exercice 3 :

Nous considérons un système de tâches pour lesquelles τ_3 précède τ_1.

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i	Priorité Π_i
τ_1	0	8	12	12	3
τ_2	1	2	3	12	2
τ_3	0	2	12	12	1

a) Tracer la simulation de l'ordonnancement produit.
b) Supposer maintenant que C_3=1, tracer la simulation.
c) Que conclure sur la viabilité de la simulation lorsqu'il existe des contraintes de précédence ?

Exercice 4 :

Nous considérons un système de tâches pour lesquelles τ_1 et τ_2 sont en exclusion mutuelle pendant toute leur durée. Nous supposons une affectation de priorités Deadline Monotonic (priorité inversement proportionnelle au délai critique).

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	0	6	16	16
τ_2	0	2	6	8
τ_3	0	6	15	16

a) Tracer la simulation de ce système.
b) Tracer une simulation lorsque C_3=5
c) Que conclure sur la viabilité de la simulation lorsqu'il existe des portions de tâches non préemptibles ?

TD VAT 3
Optimalité en FTP

La simulation, pour des systèmes de tâches indépendantes :

- Est une CNS (Condition Nécessaire et Suffisante) d'ordonnançabilité pour les systèmes de tâches périodiques ou sporadiques en monoprocesseur ;
- Est une CNS (Condition Nécessaire et Suffisante) d'ordonnançabilité pour les systèmes de tâches périodiques en multiprocesseur ;

Cependant, la simulation possède deux inconvénients majeurs :

- Dès lors qu'il existe des facteurs pratiques (exclusions mutuelles, parties non préemptibles, suspension, contraintes de précédence non consistantes avec les priorités), la simulation perd toute propriété de viabilité. Au mieux, elle peut être utilisée pour trouver un contre-exemple, ou bien une séquence hors-ligne à exécuter en-ligne, mais en aucun cas elle ne peut être utilisée pour valider un système. Elle devient dans ce cas une condition suffisante de non-ordonnançabilité (i.e. elle peut exposer un contre-exemple, mais ne l'expose pas forcément s'il existe).
- La simulation possède une complexité exponentielle, d'après (Chan & Norrish, 2016), PPCM(1, 2,…n+1) ≥ 2^n.

Plusieurs auteurs ont proposé des tests d'ordonnançabilité de complexité polynomiale. Cependant, on sait que la complexité du problème d'ordonnançabilité est Co-NP-difficile au sens fort s'il existe des dates de réveil d'une part (Leung & Whitehead, 1982), et que la complexité du problème de calcul de temps de réponse est NP-difficile au sens faible (pseudo-polynomial) même lorsque les tâches sont réveillées simultanément (Eisenbrand & Rothvos, 2008). Par conséquent, un algorithme de complexité polynomiale ne peut pas être exact. Les tests proposés sont donc généralement des heuristiques conservatives, en d'autres termes des conditions suffisantes (mais non nécessaires) d'ordonnançabilité. De plus, elles adressent une affectation de priorités particulière, en l'occurrence elles adressent les affectations de priorité optimales.

Rate Monotonic (RM), qui affecte les priorités de façon inversement proportionnelle à la période des tâches est optimale dans la classe des algorithmes à priorités fixes aux tâches lorsque (Liu & Layland, 1973) :

1. Toutes les tâches sont indépendantes préemptibles ;
2. Et le système est monoprocesseur ;
3. Et il existe un instant critique (i.e. les tâches peuvent être activées simultanément) ;
4. Et les tâches sont périodiques ou sporadiques (dans le cas sporadique, la condition 3 est forcément satisfaite) ;
5. Et Les tâches sont à échéance sur requête (i.e. $D_i = T_i$).

Un algorithme d'ordonnancement A est **optimal** dans une classe d'algorithmes d'ordonnancement Ψ, si $A \subset \Psi$ et si $\exists B \subset \Psi$ tel que B ordonnance fiablement un système S, alors A ordonnance fiablement S.

Les deux tests polynomiaux les plus connus pour RM sont :

La **borne de Liu & Layland** (Liu & Layland, 1973) : un système de n tâches indépendantes à échéances sur requête est fiablement ordonnancé par RM sur un processeur si $U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(\sqrt[n]{2} - 1\right)$. De

plus, $\lim_{n\to\infty}\left(n\left(\sqrt[n]{2}-1\right)\right) = \ln 2 \approx 0{,}693147$. Cela implique notamment qu'un système de tâches est ordonnançable (pour sûr) par RM si sa charge est inférieure ou égale à 69%.

La **borne hyperbolique** (Bini, Buttazzo, & Buttazzo, 2003) : soit la charge unitaire d'une tâche définie par $U_i = \frac{C_i}{T_i}$. Un système de n tâches indépendantes à échéances sur requête est fiablement ordonnancé par RM sur un processeur si $\prod_{i=1}^{n}(U_i + 1) \leq 2$. La borne hyperbolique est strictement plus puissante que le test de L&L, cela signifie que tout système validable par L&L est aussi validable par la borne hyperbolique, mais qu'il existe des systèmes validables par la borne hyperbolique qui ne passent pas le test de L&L. Cela rend théoriquement le test de L&L obsolète, cependant, comme on calcule toujours la charge d'un système pour vérifier qu'il n'est pas trivialement non ordonnançable, le test de L&L est immédiat, ce qui le rend tout de même intéressant à connaître.

Dans le cas où $D_i \leq T_i$, ces tests s'appliquent en remplaçant la charge U_i par la densité C_i/D_i. Ils deviennent cependant dans ce cas encore plus pessimistes que les tests à échéance sur requête.

Exercice 1 :

Soit un système de tâches indépendantes, périodiques, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	0	8	30	30
τ_2	0	6	20	20
τ_3	0	4	15	15

a) Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
b) RM donne-t-il une affectation de priorités fixes aux tâches optimale ?
c) Le système est-il démontré ordonnançable par un test de complexité polynomiale que vous connaissez ?
d) Regarder l'exercice 1 dans le TD de simulation et conclure.

Exercice 2 :

Considérons un système de tâches différées, indépendantes et périodiques, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	8	8,05	30	30
τ_2	4	4,5	20	20
τ_3	0	4,3	15	15

a) Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
b) RM donne-t-il une affectation de priorités fixes aux tâches optimale ?
c) Peut-on tout de même appliquer le test de L&L et le test hyperbolique ? Si oui, les appliquer et conclure.

Deadline Monotonic (DM), qui affecte les priorités de façon inversement proportionnelle au délai critique est optimale dans la classe des algorithmes à priorités fixes aux tâches lorsque (Leung & Whitehead, 1982) :

1. Toutes les tâches sont indépendantes préemptibles ;

2. Et le système est monoprocesseur ;
3. Et il existe un instant critique (i.e. les tâches peuvent être activées simultanément) ;
4. Et les tâches sont périodiques ou sporadiques (dans le cas sporadique, la condition 3 est forcément satisfaite) ;
5. Et Les tâches sont à **échéance contrainte** (i.e. $D_i \leq T_i$).

En fait, on peut voir RM comme le cas particulier de DM pour les tâches à échéances sur requêtes.

Exercice 3 :

Soit un système de tâches indépendantes, **sporadiques**, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	WCET C_i	Délai critique D_i	Période T_i
τ_1	4	16	16
τ_2	3	7	8
τ_3	4	6	12

a) Connaissez-vous une affectation de priorités fixes aux tâches optimale dans ce contexte ?
b) Appliquer les tests de L&L et hyperbolique en considérant la densité, et conclure.
c) Tracer une simulation du système suivant cette affectation de priorités.
d) Existe-t-il une affectation de priorités fixes aux tâches ordonnançant fiablement ce système ?

Exercice 4 :

Lorsqu'il existe des délais critiques plus grand que la période, ou bien lorsque certaines tâches sont différées et qu'il n'y a pas d'instant critique, RM et DM perdent leur optimalité. Il existe un moyen permettant de tester efficacement toute affectation de priorités possible, si le contexte satisfait 3 conditions :

- Condition 1 : l'ordonnançabilité d'une tâche dépend des tâches plus prioritaires, individuellement, mais pas de leur ordre de priorités entre elles.
- Condition 2 : l'ordonnançabilité d'une tâche dépend des tâches moins prioritaires, individuellement, mais pas de leur ordre de priorités entre elles.
- Condition 3 : si une tâche respecte ses échéances avec un niveau de priorité *i*, elle ne peut pas ne pas les respecter avec un niveau de priorité *i+1*.

Si ces 3 conditions sont satisfaites par le test d'ordonnançabilité employé, alors on peut utiliser l'algorithme d'Audsley (Audsley, 1991), dit « Optimal Priority Assignment » ou OPA.

```
T=liste des tâches (triée par exemple par ordre croissant DM)
Pour priorité Π=1 à n
      τ=tete(T)
      Prio_affectée=faux
      Répéter
            Si τ ne respecte pas ses échéances avec priorité Π
                  Si τ=last(T) -- on a épuisé toute la liste
                        Renvoyer non ordonnançable
                  Sinon τ=next(T)
            Sinon
                  Prio_affectée=vrai
                  Assigner priorité Π à τ et remove(T,τ)
      Jusqu'à Prio_affectée
Fait
```

a) Quelle est la complexité de cet algorithme en supposant que vérifier qu'une tâche respecte ses échéances ait une complexité O(1) ?
b) Supposons que nous ayons 25 tâches, combien au maximum de test d'ordonnançabilité de tâche devrons-nous mener ?

Une façon de tester si une tâche respecte ses échéances, indépendamment des autres priorités, consiste à simuler (si on se trouve dans un contexte ou la simulation est viable) le système en affectant :

- La priorité Π à τ
- La priorité $\Pi+1$ à toutes les autres tâches encore présentes dans T
- La priorité déjà choisie pour les tâches affectées précédemment

On ne regarde alors que pour la tâche τ considérée si les échéances sont respectées (on ne regarde pas si les échéances des tâche plus prioritaires sont respectées ou non).

c) En supposant que le test d'ordonnançabilité de tâche retenu soit la simulation, est-ce que, si les tâches sont non préemptibles, on peut utiliser OPA ?
d) Observer le système de tâches sporadiques indépendantes et préemptibles suivant. DM est-il optimal dans ce cadre ? OPA s'applique-t-il en considérant la simulation pour valider l'ordonnançabilité des tâches ? Le système est-il ordonnançable en priorités fixes aux tâches ?

	WCET C_i	**Délai critique D_i**	**Période T_i**
τ_1	4	16	16
τ_2	2	9	8
τ_3	5	13	12

Bibliographie

Audsley, N. C. (1991). *Optimal priority assignment and feasibility of static priority tasks with arbitrary start times.* Computer Science Report YCS 164, York.

Bini, E., Buttazzo, G. C., & Buttazzo, G. M. (2003). Rate monotonic analysis: the hyperbolic bound. *IEEE Transactions on Computers 52(7)*, 933-942.

Chan, H.-L., & Norrish, M. (2016). Proof pearl: bounding least common multiples with triangles. *International Conference on Interactive Theorem Proving*, (pp. 140-150).

Eisenbrand, F., & Rothvos, T. (2008). Static-priority real-time scheduling: Response time computation is NP-hard. *Real-Time Systems Symposium*, (pp. 397-406).

Leung, J. Y.-T., & Whitehead, J. (1982). On the complexity of fixed-priority scheduling of periodic, real-time tasks. *Performance evaluation*, 237-250.

Liu, C. L., & Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM 20 (1)*, 46-61.

TD VAT 4
Analyse de temps de réponse- RTA

Jusqu'à présent nous avons utilisé la simulation, de complexité exponentielle pour valider des systèmes de tâches. Nous avons vu que dans certains cas très restreints, il existait des conditions suffisantes de complexité polynomiale applicables.

Cependant, nous avons vu lors du dernier TD que la complexité de calcul de pire temps de réponse est NP-difficile au sens faible, par conséquent, il devrait exister un algorithme de calcul de temps de réponse de complexité pseudo-polynomiale.

Période d'activité de niveau i : dans un ordonnancement à priorités fixes aux tâches, une période d'activité de niveau i représente une occupation continue du processeur en ne considérant que les tâches de priorité ≥ i. Une période d'activité commence et se termine sur un point creux.

Point creux : instant pendant lequel le processeur n'a aucune charge à traiter.

La Figure 1 représente des périodes d'activité de niveau 1 (plus basse priorité) de l'ordonnancement du système de deux tâches périodiques simultanées $\{\tau_1 <C_1=1, T_1=4>, \tau_2 <C_2=10, T_2=14>\}$. Noter que les périodes d'activité sont indépendantes des échéances.

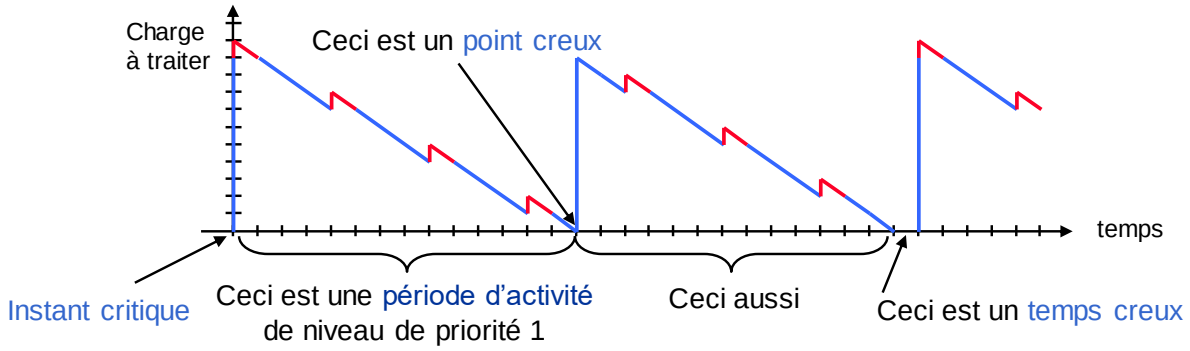


Figure 1 : Périodes d'activité, points creux et temps creux

A chaque réveil, on observe une montée de la charge à traiter (à chaque réveil de τ_i, la charge à traiter augmente de C_i). A chaque unité de temps, le processeur traite une unité de charge, si celle-ci est non nulle.

Théorème de l'instant critique : étant donné un ensemble de tâches ordonnancées en priorités fixes aux tâches, la plus longue période d'activité s'observe à l'instant critique.

Théorème (Audsley, 1991) : le plus long temps de réponse d'une tâche de priorité i se trouve dans la plus longue période d'activité de niveau i.

Sur la Figure 1, la première période d'activité dure 14 unités de temps, la seconde en dure 13. Supposons que la priorité de τ_2 soit égale à 1, cela signifie qu'elle se termine « en dernier », c'est-à-dire au point creux marquant la fin de chaque période d'activité. Elle se termine donc à l'instant 14 lors de son instance démarrée à l'instant critique, sa première instance possède donc un temps de réponse de 14 unités de temps. Sa seconde instance est activée à l'instant 14 et termine à l'instant 27,

ce qui lui donne un temps de réponse de 13 unités de temps. Ensuite, le système répète le cycle [0..28[à l'infini. Nous pouvons observer cela sur une simulation donnée sur la Figure 2.

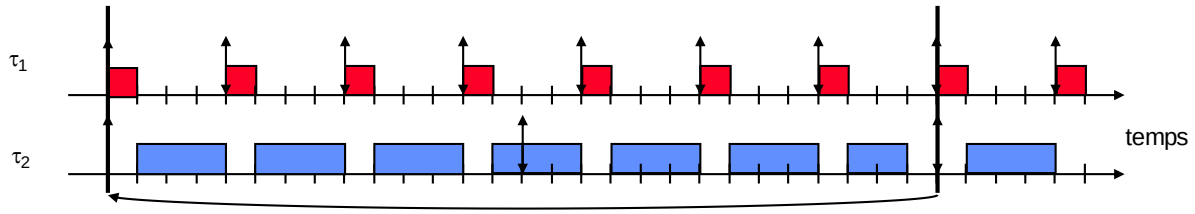


Figure 2 : Simulation de l'ordonnancement RM du même système

Pour le moment, tracer les périodes d'activité et simuler revient au même, cependant, il y a ici un premier gain : d'après le théorème d'Audsley, s'il existe un instant critique, c'est forcément dans la période d'activité initiée par l'instant critique que l'on trouve le plus long temps de réponse. On pourrait s'arrêter ici de construire la période d'activité lorsque l'on trouve le point creux à l'instant 14. En effet, dans la seconde période d'activité, le temps de réponse trouvé ne peut pas être plus long.

Cela reste cependant de complexité exponentielle, la période d'activité pouvant au pire durer le PPCM des périodes si U=1. D'après (Eisenbrand & Rothvos, 2008), on doit pouvoir trouver un algorithme plus efficace que la construction systématique de toute la période d'activité.

L'idée proposée par (Joseph & Pandya, 1986) est la suivante : soit une tâche τ_i dont on veut calculer le pire temps de réponse, et soit hp(i) l'ensemble des indices de tâches plus prioritaires que (ou ex-aequo avec) la tâche étudiée τ_i. Nous allons tenter d'établir de façon efficace la date de fin de période d'activité initiée par un instant critique ayant eu lieu à la date 0.

La période d'activité de niveau de priorité de τ_i dure au moins C_i, la pire durée de τ_i. Nous notons cette durée $w_i^0 = C_i$.

Puisque la période d'activité initiée par l'instant critique dure au moins w_i^0, toute tâche plus prioritaire activée dans l'intervalle [0.. w_i^0[va venir préempter la tâche τ_i, rallongeant ainsi la période d'activité (comme les « pointes » observées à chaque réveil de τ_i sur la Figure 1). Su l'intervalle [0..t[, le nombre de réveils d'une tâche j est donné par $\left\lceil \frac{t}{T_j} \right\rceil$, à chacun de ces réveils, la période d'activité est prolongée de C_j. Par conséquent, si la période d'activité dure au moins w_i^0, alors elle dure au moins :

$$w_i^1 = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^0}{T_j} \right\rceil C_j$$

Mais si elle dure au moins w_i^1, alors tout réveil d'une tâche plus prioritaire sur [0.. w_i^1[vient allonger cette période d'activité. On voit que l'on recherche en fait le plus point fixe non nul w_i^* tel que :

Response-Time Analysis (RTA): $w_i^* = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^*}{T_j} \right\rceil C_j$

w_i^* donne la date de fin de la première instance de la tâche τ_i initiée par l'instant critique. Le pire temps de réponse de la première instance de cette tâche, activée à l'instant 0, est donc $TR_i(1) = w_i^*$.

- Si ce temps de réponse n'est pas supérieur à la période de τ_i, alors il correspond à la date de fin de la période d'activité, qui ne contient par conséquent qu'une instance de τ_i. D'après le théorème d'Audsley, c'est donc le pire temps de réponse de la tâche et $TR_i = TR_i(1)$. La complexité de ce calcul de point fixe est pseudo-polynomiale.

- Si $w_i^* > T_i$, nous n'avons pas fini. Cela signifie que la seconde instance de τ_i appartient aussi à la période d'activité la plus longue. Par conséquent, cette instance peut aussi être celle qui possède le plus long temps de réponse, et il faut donc l'étudier.

C'est ce que propose (Lehoczky, 1990) en proposant une formule généralisée du calcul de RTA :

$k = 0$
Répéter
    $k = k + 1$
    $w_i^0(k) = kC_i$
    Trouver le plus petit point fixe non nul $w_i^*(k) = kC_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^*(k)}{T_j} \right\rceil C_j$
    $TR_i(k) = w_i^*(k) - (k-1)T_i$
Jusqu'à $TR_i(k) \leq T_i$
$TR_i = \max_{j=1..k}\left(TR_i(j)\right)$

Cette formule est **à connaître sur le bout des doigts**, puisqu'elle est à la base de l'analyse de temps de réponse en priorités fixes aux tâches. C'est une formule de calcul exacte de temps de réponse, de complexité pseudo-polynomiale. Elle donne une **condition nécessaire et suffisante** d'ordonnançabilité lorsqu'il existe (ou peut exister) un instant critique, puisqu'il suffit de vérifier que pour toute tâche, $TR_i \leq D_i$.

Remarquer que pour k=1, cette formule est celle de (Joseph & Pandya, 1986).

Nous verrons par la suite des extensions de cette formule pour prendre en compte les exclusions mutuelles et les contraintes de précédence.

Exercice 1

Lien entre formule RTA et représentation de la période d'activité.

Observer la période d'activité du système de deux tâches {τ_1 <C_1=1, T_1=4>, τ_2 <C_2=10, T_2=14>} représentée sous forme de fonction escalier, appelée RBF (pour Request Bound Function) sur la Figure 3.

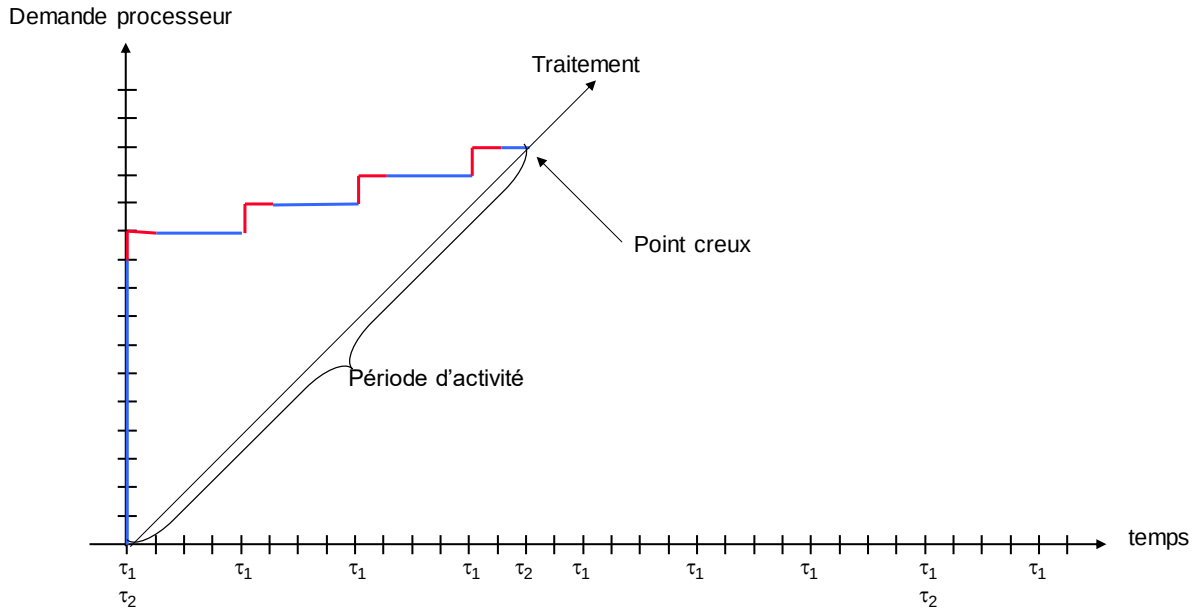


Figure 3 : Période d'activité représentée sous forme de RBF

Calculer avec la RTA le pire temps de réponse de la tâche τ_2, et observer la représentation graphique des pas de calcul la Figure 4.

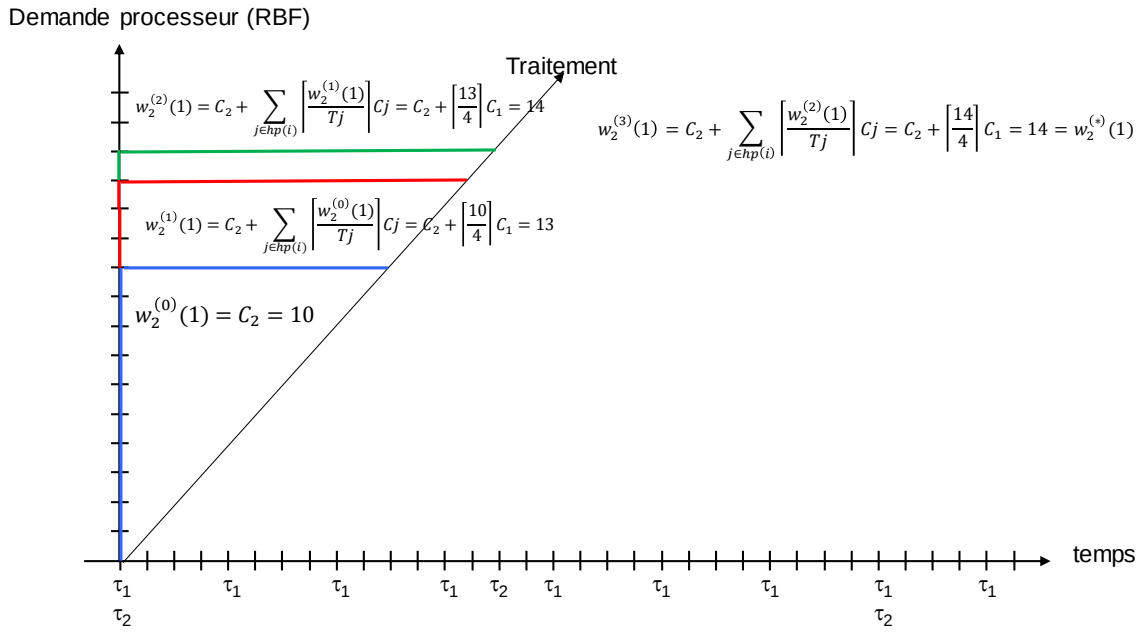


Figure 4 : Valeurs obtenues par le calcul RTA représentées sur la RBF

Le calcul RTA permet donc de calculer de façon efficace quelle est l'intersection entre la RBF et la puissance du processeur, représentée par la droite y=x.

Exercice 2

Soit le système S1 de trois tâches indépendantes, périodiques, simultanées, à échéances sur requête, dont les paramètres <C_i, T_i> sont donnés par {<2,4>, <1,4>, <1,8>}, ordonnancé suivant RM.

a) Calculer la charge du système S1, et conclure.
b) RM est-il optimal pour S1 ?
c) La RTA est-elle exacte, pessimiste ou optimiste ?
d) Calculer avec la RTA le temps de réponse de chaque tâche, et conclure.

Exercice 3

Soit le système S2 de trois tâches indépendantes, sporadiques, à échéances sur requête, dont les paramètres <C_i, T_i> sont donnés par {<2,7>, <2,4>, <3,14>}, ordonnancé suivant RM.

a) Calculer la charge du système S2, et conclure.
b) RM est-il optimal pour S2 ?
c) La RTA est-elle exacte, pessimiste ou optimiste ?
d) Calculer avec la RTA le temps de réponse de chaque tâche, et conclure.

Exercice 4

Soit le système S3 de trois tâches indépendantes, périodiques, à échéances sur requête, à départs différés, dont les paramètres <O_i, C_i, T_i> sont donnés par {<0,1,4>, <1,3,6>, <2,1,4>}, ordonnancé suivant RM.

a) Calculer la charge du système S3, et conclure.

b) RM est-il optimal pour S3 ?
c) La RTA est-elle exacte, pessimiste ou optimiste ?
d) Calculer avec la RTA le temps de réponse de chaque tâche, et conclure.
e) La simulation est-elle exacte, pessimiste ou optimiste pour S3 ?
f) Tracer une simulation et comparer les pires temps de réponse observés par rapport à ceux obtenus par la RTA. Comment expliquer les différences observées ?

Exercice 5

Soit le système S4 de deux tâches indépendantes, sporadiques, dont les paramètres < C_i, D_i, T_i> sont donnés par {<26,26,70>, <62,118,100>}, ordonnancé suivant DM. Cet exemple est tiré de (Lehoczky, 1990) et a permis d'illustrer le fait que ce n'est pas forcément la première instance qui possède le plus long temps de réponse, mais l'une des instances se trouvant dans la plus longue période d'activité.

a) Calculer la charge du système S3, et conclure.
b) DM est-il optimal pour S4 ?
c) La RTA est-elle exacte, pessimiste ou optimiste ?
d) Calculer avec la RTA le temps de réponse de la première instance de τ_2.
e) Calculer avec la RTA le temps de réponse de la seconde instance de τ_2.
f) En réalité, on trouve 7 instances de la tâche τ_2 dans la période d'activité initiée par l'instant critique, et c'est la 5ème instance qui possède le plus long temps de réponse. Calculer ce temps de réponse.

Exercice 6

En observant la RBF donnée sur la Figure 3, qu'adviendrait-il de la durée de la période d'activité si :

a) Une durée d'exécution de tâche était moins longue ?
b) Une période était plus longue que prévue ?
c) Une date de réveil était plus grande que 0 ?
d) Peut-on en déduire que la RTA est C-viable, r-viable, et T-viable ?

Bibliographie

Audsley, N. C. (1991). *Optimal priority assignment and feasibility of static priority tasks with arbitrary start times.* Computer Science Report YCS 164, York.

Eisenbrand, F., & Rothvos, T. (2008). Static-priority real-time scheduling: Response time computation is NP-hard. *Real-Time Systems Symposium*, (pp. 397-406).

Joseph, M., & Pandya, P. (1986). Finding response times in a real-time system. *The Computer Journal 29 (5)*, 390-395.

Lehoczky, J. P. (1990). Fixed priority scheduling of periodic task sets with arbitrary deadlines. *Proc. of the 11th Real-Time Systems Symposium*, (pp. 201-209).

TD VAT 5
Analyse de temps de réponse de tâches dépendantes

Exclusions mutuelles

Nous avons évoqué le problème d'**inversion de priorité** : sans modification de la priorité de tâches partageant des ressources, tout se passe comme si une tâche prioritaire attendant une ressource détenue par une tâche moins prioritaire se voyait hériter de la priorité de cette dernière. Ainsi, des tâches de priorité intermédiaires s'exécuteraient avant la tâche prioritaire bloquée. Ceci est illustré sur la Figure 1.

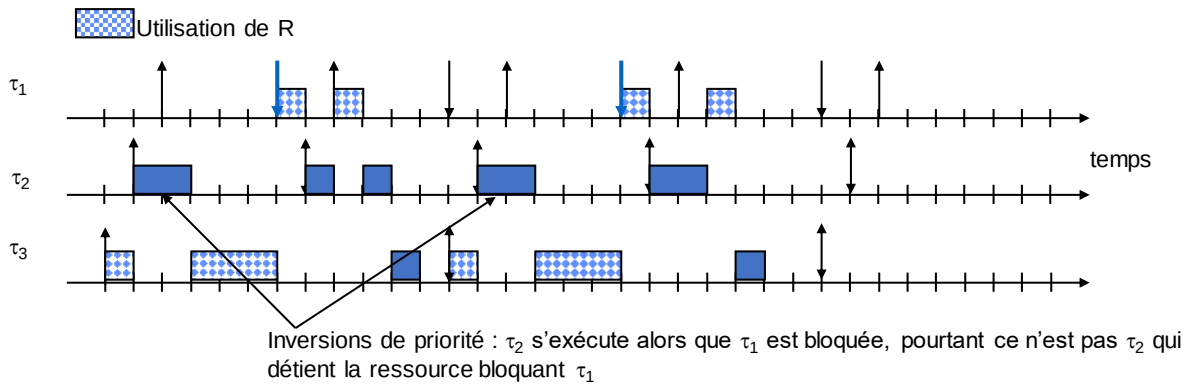


Figure 1 : Inversion de priorité

Deux protocoles de gestion de ressources sont proposés dans (Sha, Rajkumar, & Lehoczky, 1990), le **protocole à priorité héritée** (PIP – Priority Inheritance Protocol), qui consiste, lorsqu'une tâche cherche à prendre un mutex qui est déjà détenu par une autre tâche, à faire temporairement hériter de sa priorité la tâche détenant la ressource. Ce protocole est illustré sur la Figure 2.

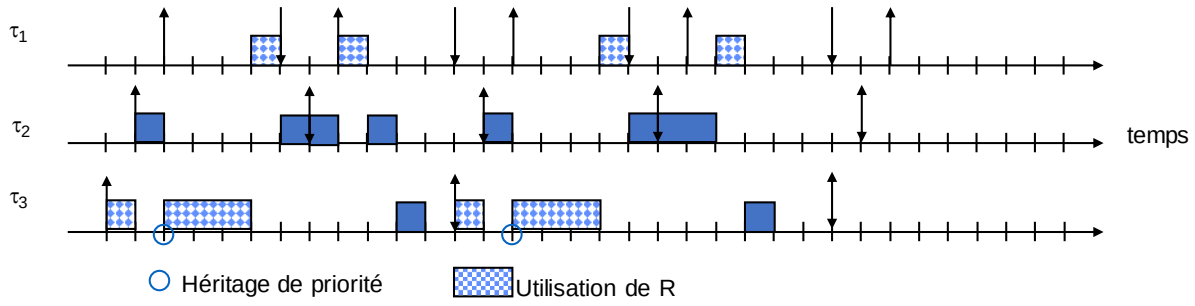


Figure 2 : Protocole à priorité héritée

Dans ce cas, on évite l'inversion de priorité, cependant :

- Il y a possibilité d'interblocage (voir problème des philosophes) ;
- Une tâche devant prendre plusieurs mutex peut se trouver bloquer par une section critique moins prioritaire pour chaque mutex (voir Figure 3).

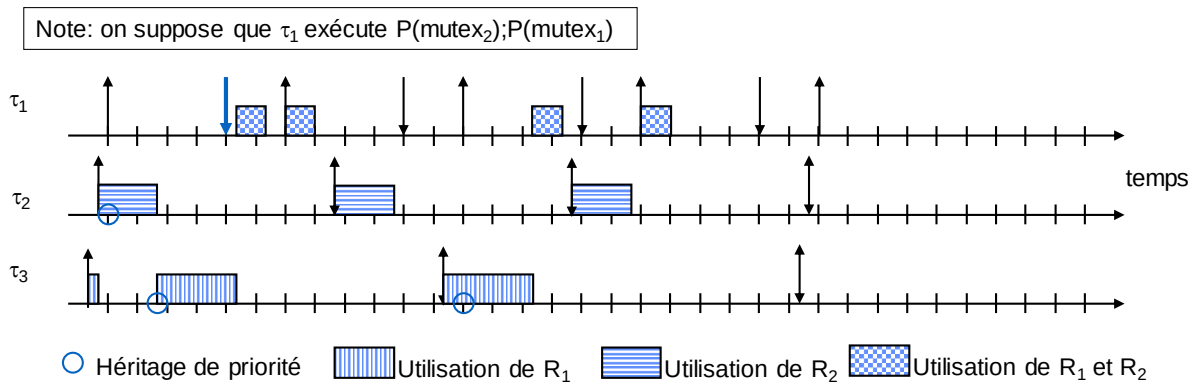


Figure 3 : Avec le protocole à priorité héritée, chaque demande de mutex peut donner lieu à attente de fin de section critique

C'est pour cela que ce même article propose un **protocole à priorité plafond** (PCP - Priority Ceiling Protocol) : chaque mutex se voit doter d'une priorité dite plafond, égale à la priorité maximale des tâches susceptibles de prendre le mutex. Le système se voit muni d'une valeur de plafond, égale à la priorité plafond maximale des mutex en cours d'utilisation. La prise d'un mutex n'est accordée que si (1) il est libre, et (2) la priorité de la tâche le demandant est strictement supérieure au plafond système, ou le plafond système est dû à la tâche.

Le protocole à priorité plafond assure :

- L'absence d'interblocage (appliquer le PCP à l'exemple des philosophes) ;
- Une tâche accédant à plusieurs mutex ne peut être bloquée au plus que par une seule section critique de priorité inférieure (voir Figure 4).

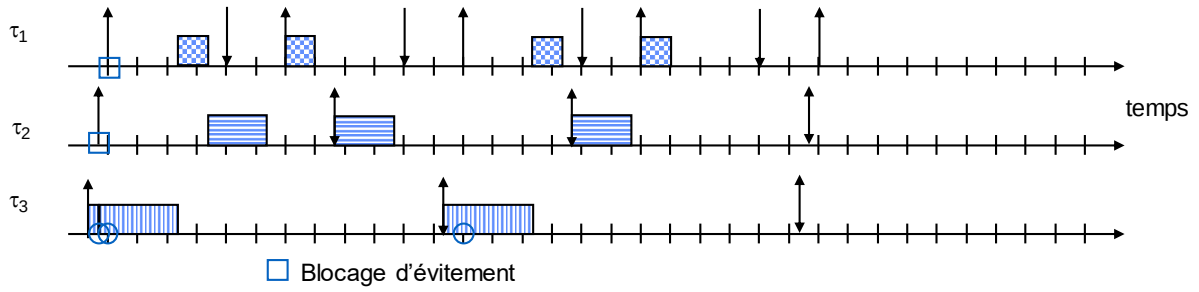


Figure 4 : Le protocole à priorité plafond garantit qu'au plus une section critique de priorité inférieure retarde une tâche

Dans la réalité, les RTOS proposant le protocole à priorité plafond en font une implémentation ayant les mêmes propriétés pire cas, mais beaucoup plus simple : le protocole à priorité plafond immédiat. Une tâche obtenant le mutex hérite directement de la priorité plafond du mutex. Sur l'exemple de la Figure 4, le comportement est le même en apparence (voir Figure 5), bien que l'héritage de priorité ait lieu en début de chaque section critique : dans ce cas, la tâche entrant en section critique hérite directement du plafond des ressources, soit la priorité de τ_1. Cela n'est cependant pas toujours le cas, ainsi, si une tâche indépendante de priorité intermédiaire entre celle de τ_1 et celle de τ_3 s'activait entre l'activation de τ_3 et l'activation de τ_1, celle-ci s'exécuterait avec PCP, alors qu'elle ne s'exécuterait pas en Immediate-PCP. Noter que ce protocole avait été proposé dans (Kaiser, 1982), mais que cet article français, paru 8 ans avant (Sha, Rajkumar, & Lehoczky, 1990), n'est malheureusement quasiment jamais cité.

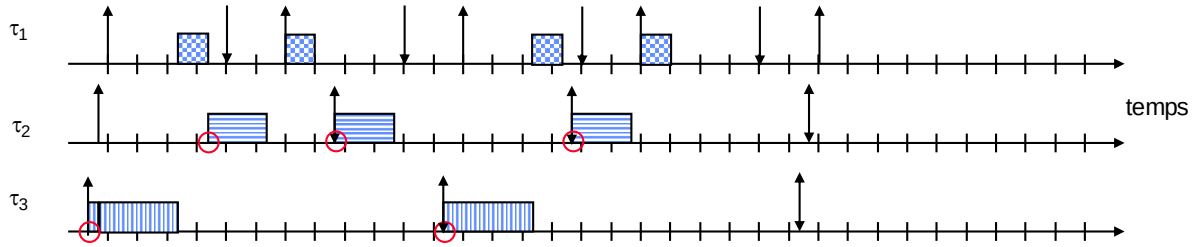


Figure 5 : Version immédiate du protocole à priorité plafond

Attention, dans ces trois protocoles, même une tâche n'utilisant pas de ressource peut se voir bloquer par une section critique de priorité inférieure, si celle-ci accède à un mutex dont le plafond est supérieur ou égal à la priorité de la tâche considérée, à cause de l'héritage de priorité.

Rappelons que la simulation n'est plus C-viable, T-viable ou r-viable avec des ressources critiques, et qu'il n'existe pas d'affectation de priorité optimale. Comment valider un tel système ?

En faisant une RTA modifiée : il suffit de considérer un pire cas qui consiste à supposer que, lors de la période d'activité initiée par l'instant critique, la (dans le cas PCP) ou les (dans le cas PIP) sections critiques les plus longues de tâches moins prioritaires que la tâche étudiée viennent juste de commencer. On suppose donc qu'il faudra que la tâche étudiée attende la pire durée de blocage due aux sections critiques de priorité inférieure démarrées « juste au pire moment ». Notons que la RTA ainsi obtenue devient une condition suffisante (mais non nécessaire) d'ordonnançabilité, car il est possible que le pire cas ne puisse jamais survenir, notamment lorsque les tâches sont périodiques. Nous ne cherchons pas à avoir une méthode exacte, car dans le cas de ressources, nous savons que le problème d'ordonnançabilité est NP-difficile au sens fort (Mok, 1983). Cette pire durée de blocage de la tâche τ_i par des sections critiques de priorité inférieure se note B_i. Dans la RTA, il suffit alors d'ajouter ce terme :

$$w_i^0(k) = B_i + kC_i$$

$$w_i^*(k) = B_i + kC_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^*(k)}{T_j} \right\rceil C_j$$

Comment calculer ce terme B_i ? C'est assez simple, voyons cela sur un exemple : nous considérons une table (voir Figure 6) dans laquelle nous plaçons la tâche qui nous intéresse sur une ligne, les tâches plus prioritaires ou ex-aequo au-dessus, les tâches moins prioritaires au-dessous. Nous plaçons une ressource (de façon équivalente un mutex) sur chaque colonne, et plaçons une durée de section critique ligne i colonne j si la tâche i possède une section critique de cette durée utilisant la ressource j. Ainsi sur l'exemple, la tâche 1 utilise la ressource 2 pendant 2 unités de temps.

Traçons un trait sous la ligne de la tâche étudiée, ici la tâche 2. Toute section critique possédant une valeur au-dessus et au-dessous du trait est susceptible de voir une section critique de priorité inférieure retarder la tâche étudiée. Par exemple, ici, la section critique de la tâche 4 sur la ressource 1 peut hériter d'une priorité supérieure ou égale à la priorité de la tâche 2, et ainsi retarder celle-ci.

Dans le cas PIP, chaque section critique concernée peut retarder la tâche une fois (elle peut avoir débuté juste avant l'instant critique), donc $B_{2,PIP}$=max(3)+max(1)+max(2,4)=8. Dans le cas PCP ainsi qu'Immediate-PCP, à cause du principe du plafond, au plus une seule section critique apparaissant au-dessus et au-dessous du trait a pu commencer, par conséquent $B_{2,PCP}$=max(3,1,2,4)=4.

	R_1	R_2	R_3	R_4	R_5	R_6
τ_1		2				5
τ_2	2					
τ_3		1	2			
τ_4	3			2		2
τ_5			4	3		
τ_6						
τ_7			6			4

Niveau de priorité

Figure 6 : Calcul du facteur de blocage B_2

Exercice 1

Calculer pour chaque tâche de la Figure 6 le facteur de blocage PCP, et le facteur de blocage PIP de chaque tâche.

Exercice 2

Proposer une adaptation du test hyperbolique et du test de L&L aux tâches partageant des ressources. Justifier le fait que le test ainsi adapté reste une condition suffisante d'ordonnançabilité.

Exercice 3

Soit le système de tâches sporadiques à échéance sur requête exprimé en AADL sur la Figure 7. La communication par data port est implémentée par un tableau noir, protégé par un mutex. Nous supposons que les sections critiques d'accès à ce tableau noir utilisent, aussi bien en lecture qu'en écriture, 0.5 unité de temps. L'implémentation est en POSIX, et un protocole de gestion de ressources a été choisi afin d'éviter l'inversion de priorité.

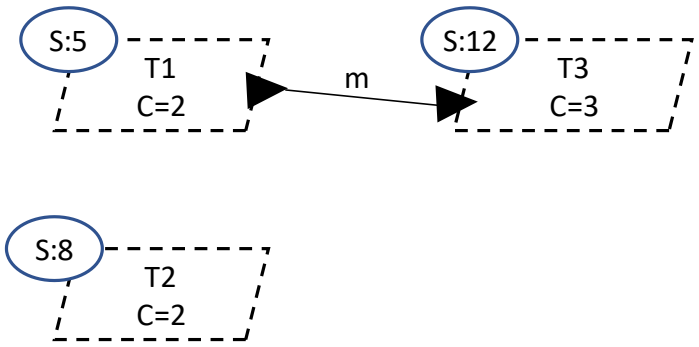


Figure 7 : Système à valider

a) Quel est le protocole de gestion de ressources choisi ?
b) Existe-t-il une affectation de priorités optimale dans ce contexte ? Si oui, la choisir, sinon, en choisir une qui a de bonnes chances de fonctionner.
c) Calculer pour chaque tâche son facteur de blocage.

d) Calculer le pire temps de réponse de chaque tâche, et conclure sur l'ordonnançabilité du système.
e) Représenter par une simulation le comportement du système en supposant que les tâches sont réveillées périodiquement, et que T3 se éveille initialement à l'instant 0, T2 à l'instant 0.1 et T1 à l'instant 0.2.

Contraintes de précédence

Lorsqu'il existe des **contraintes de précédence**, il nous faut intégrer un nouveau paramètre caractérisant le retard potentiel que peut prendre une tâche par rapport à sa « période escomptée ». En effet, considérons la Figure 8, sur laquelle T1 communique avec T2 via event data port. Globalement, la période de T2 est, comme T1, de 10 unités de temps. Cependant, entre 2 activations successives de T2, il peut y avoir moins de 10 unités de temps : si lors d'une exécution, en supposant que le message est envoyé par T1 en fin d'exécution, le temps de réponse de T1 vaut 3 unités de temps, mais que dans sa prochaine exécution le temps de réponse de T1 vaut presque 0 unités de temps, alors entre la première et la seconde activation de T2, seules 7 unités de temps se sont écoulées.

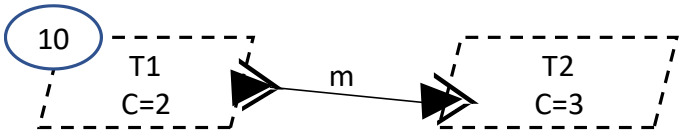


Figure 8 : Communication synchrone impliquant une contrainte de précédence T1 précède T2

Globalement la période de T2 est bien de 10 unités de temps, cependant elle peut être amenée à subir un retard dans sa période qui est égal au pire temps de réponse de T1. Ce retard est appelé gigue d'activation (en anglais *jitter*), et est dénoté J_i. Ici J_1=0, et J_2=TR(τ_1).

Théorème de l'instant critique avec gigue (Audsley, 1991) : la plus longue période d'activité pour des tâches avec gigue survient lorsque toutes les tâches sont activées simultanément après avoir subi leur pire gigue, puis exécutées périodiquement sans gigue.

Nous pouvons donc adapter la formule de calcul RTA au cas des gigues, tout simplement en considérant un instant critique à l'instant 0, mais que toutes les tâches τ_i considérées dans le calcul de la période d'activité sont activées à la date $-J_i$. Cela revient, dans la RBF, lorsque $J_i<T_i$, à avoir la première « marche » à l'instant 0 (comme sans gigue), mais la seconde à la date T_i-J_i, puis les autres espacées de T_i unités de temps. Ainsi, sur la Figure 9, on voit que pour J_1=1, le second réveil de τ_1 se produit à la date 3, puis les autres périodiquement de période 4, aux dates 7, 11, 15, etc. Cependant, pour J_1=3, le second réveil de τ_1 se produit à la date 1, puis les autres aux dates 5, 9, 13, etc., ce qui augmente grandement la durée de la période d'activité.

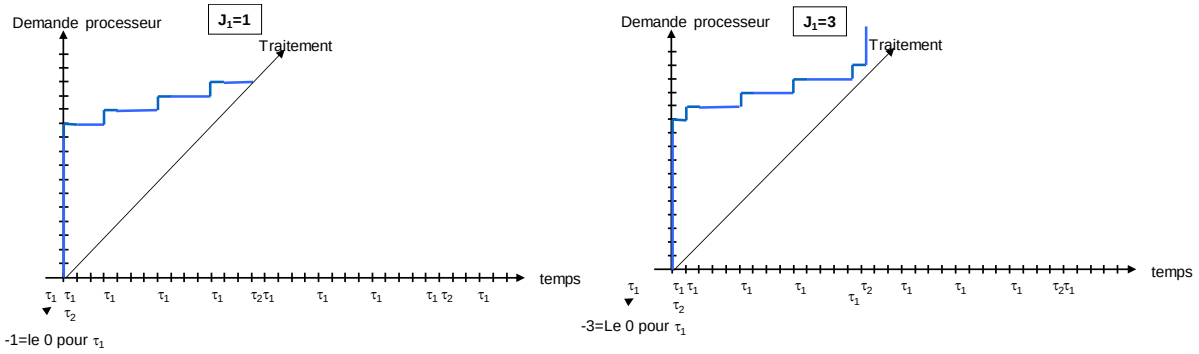


Figure 9 : RBF comparées de {τ_1 <C_1=1, J_1=1 ou 3, T_1=4>, τ_2 <C_2=10, J_2=0, T_2=14>}

Graphiquement, on peut voir que la RTA est J-viable, i.e., le temps de réponse ne peut pas se voir rallonger en réduisant la gigue.

La formule de calcul RTA devient donc :

$k = 0$
Répéter
$\quad k = k + 1$
$\quad w_i^0(k) = B_i + kC_i$
$\quad$ Trouver point fixe non nul $w_i^*(k) = B_i + kC_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^*(k)+J_j}{T_j} \right\rceil C_j$
$\quad TR_i(k) = w_i^*(k) - (k-1)T_i + J_i$
Jusqu'à $TR_i(k) \leq T_i$
$TR_i = \max_{j=1..k}\left(TR_i(j)\right)$

Nous voyons que la gigue apparaît à deux endroits du calcul : dans la recherche du point fixe, le nombre de réveils d'une tâche τ_j ayant une gigue est non plus calculé sur [0..t[mais sur [-J_j..t[d'où l'ajout de J_j dans le numérateur de la partie entière supérieure.

De plus, la tâche τ_i étudiée, comme les autres, est supposée être réveillée non pas à l'instant 0, mais à l'instant -J_i puisqu'elle subit sa gigue maximale. Par conséquent, on ajout J_i au temps de réponse, puisque $w_i^*(k)$ nous donne la date de fin de période d'activité, mais que la date d'activation de la tâche étudiée n'est pas 0 mais -J_i.

Exercice 4

Soit le système de tâches à échéance sur requête exprimé en AADL sur la Figure 10. La communication par data port est implémentée par un tableau noir, protégé par un mutex. Nous supposons que les sections critiques d'accès à ce tableau noir utilisent, aussi bien en lecture qu'en écriture, 0.5 unité de temps. L'implémentation est en VxWorks, et un protocole de gestion de ressources a été choisi afin d'éviter l'inversion de priorité. La communication par message utilise des messageQueues, gérant le buffer de message en exclusion mutuelle avec le même protocole de gestion de ressource, et nous supposons qu'envoyer ou lire un message existant prend au pire 0,6 unités de temps en exclusion mutuelle.

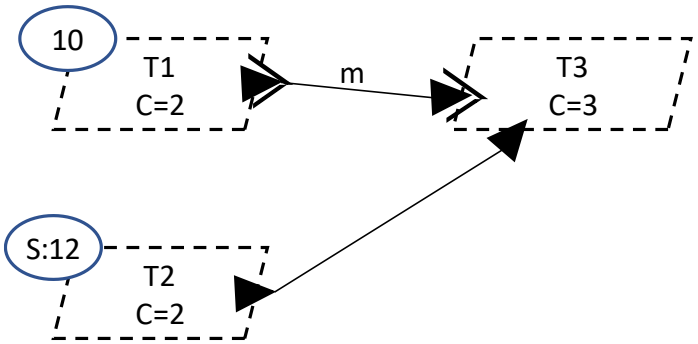


Figure 10 : Système avec exclusion mutuelle et précédence à valider

a) Quel est le protocole de gestion de ressources choisi ?
b) Existe-t-il une affectation de priorités optimale dans ce contexte ?

c) Si nous choisissions une priorité de T3 supérieure à celle de T1, que se passerait-il pour le calcul de la gigue J_3 ? En déduire une affectation de priorités qui nous permettra d'éviter cela.
d) Calculer pour chaque tâche son facteur de blocage.
e) Calculer le pire temps de réponse de chaque tâche, et conclure sur l'ordonnançabilité du système.

Bibliographie

Audsley, N. C. (1991). *Optimal priority assignment and feasibility of static priority tasks with arbitrary start times.* Computer Science Report YCS 164, York.

Kaiser, C. (1982). Exclusion mutuelle et ordonnancement par priorité. *Technique et science informatiques 1 (1)*, 59-68.

Mok, A. K.-L. (1983). *Fundamental design problems of distributed systems for the hard-real-time environment.* thesis, Massachusetts Institute of Technology.

Sha, L., Rajkumar, R., & Lehoczky, J. P. (1990). Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on computers 39 (9)*, 1175-1185.

Validation temporelle par Model Checking

Emmanuel GROLLEAU

Motivations

- Quel que soit le contexte industriel, l'informatique critique répond à des standards de sûreté de fonctionnement
  - Avionique DO-178B/C, DO-254: Design Assurance Level DAL A/B/C/D/E
  - Automobile ISO 26262: Automotive Design Insurance Level ASIL 4/3/2/1/QM
  - Ferroviaire/autre: CENELEC 20126/128/128: Software Insurance Level SIL 4/3/2/1/(0)
  - Drone: Joint Authorities for Rulemaking of Unmanned Systems (JARUS) définit AMC 1309 – en cours d'élaboration

Sûreté de fonctionnement

- Des méthodes sont mises en place pour garantir au mieux la sûreté
- On peut faire des simulations, puis des centaines ou milliers d'heures de tests => coût élevé et fiabilité basse
- Trois grandes familles de méthodes de vérification formelle
  - **Vérification par modèle** (*model checking – RdP, automate temporisé, TLA+, etc.*)
  - Preuve assistée (*theorem proving – Coq, Isabelle, etc.*)
  - Raffinement de spécification (méthode B)

Carl Adam Petri, 1962

RÉSEAUX DE PETRI

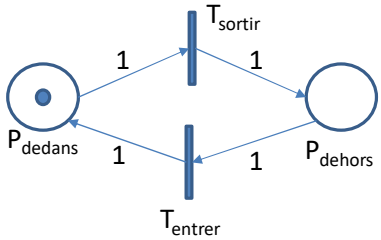
Modèle

- Un RdP est un modèle
  - Modèle = abstraction représentative de ce qui est modélisé
  - En informatique, et V&V, permet des analyses et preuves permettant d'augmenter la confiance dans un système
  - Généralement notation mathématique (ou graphique pour lequel une équivalence mathématique existe)
- Un RdP classique ne représente pas ce qui sera, mais ce qui peut être
- De base, il n'a pas un comportement temporel ou événementiel déterministe. Lorsqu'une transition est franchissable, elle n'est pas forcée, elle peut être franchie (on ne dit pas quand), ou ne pas l'être

Expressivité d'un modèle

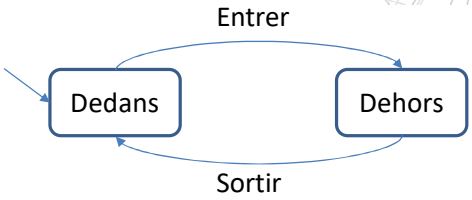
- Machine de Turing : puissance d'expression d'un programme exécuté sur architecture von Neumann
- Réseau de Petri : sait compter, mais ne sait pas représenter un test à 0 pour une place qui compte
- Automate fini : ne sait représenter que les langages réguliers (représentables par expression régulière), ne sait pas compter, ne sait pas représenter $a^n b^n$
- Plus de puissance = plus de complexité d'analyse

Représentation graphique d'un RdP



RdP

Equivalence



Automate fini

jeton

place

transition

n → **arc** valué

[1,0] : **marquage initial**

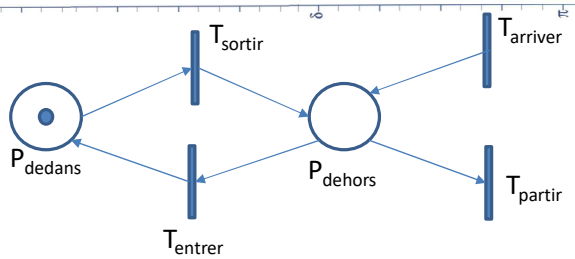
T_{sortir} est **franchissable** (ou **tirable**)

le **franchissement** de T_{sortir} consomme un jeton dans P_{dedans} et produit un jeton dans P_{dehors}. Après franchissement le marquage est [0,1].

Un RdP est un graphe **bipartite**

Ce RdP est **sauf**, par construction, chaque place ne peut contenir qu'un jeton au plus.

Quelques exemples simples



$T_{arriver}$ est une transition source (pas de précondition)

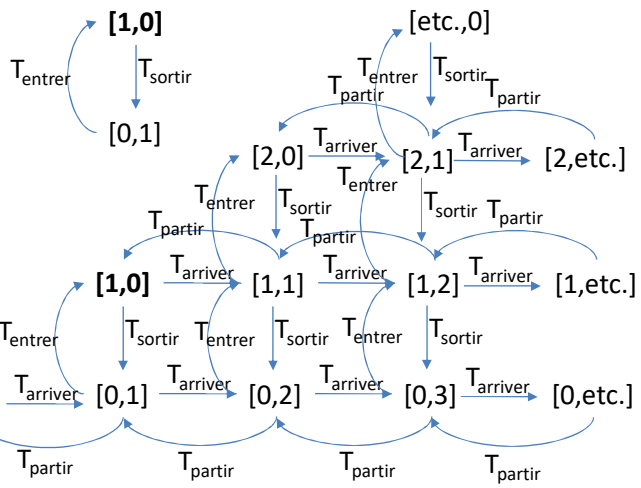
T_{partir} est une transition puits (pas de postcondition)

Ce RdP est **non borné**. Potentiellement certaines places (ici toutes) peuvent contenir une infinité de jetons

Le marquage M=[0,0] est-il atteignable ? Si oui, exhiber une séquence de franchissements de transitions y menant.

Graphe des marquages

- Dans le cas borné, le GM est borné



- Dans le cas non borné, il ne l'est pas

Formalisation

- Réseau de Petri : N=(P,T,W) où
  - P ensemble fini de places
  - T ensemble fini de transitions $(P \cap T)=\varnothing$
  - W: $P \times T \cup T \times P \rightarrow \mathbb{N}$ fonction de valuation
    - ✓Pré $W_{|P \times T}$ fonction de précondition
    - ✓Post $W_{|T \times P}$ fonction de postcondition
- Etat d'un RdP : fonction de marquage $M : P \rightarrow \mathbb{N}$, ou vecteur de marquage $M \in \mathbb{N}^{|P|}$
- Marquage initial : $M_0 : P \rightarrow \mathbb{N}$

Comportement

- $t \in T$ est franchissable à partir du marquage M, noté **M(t>**
  - Ssi $\forall p \in P$, $M(p) \geq W(p,t)$
- $t \in T$ est franchie à partir du marquage M, pour donner M', noté **M(t>M'**
  - Si M(t>
  - $\forall p \in P$, $M'(p)=M(p)-W(p,t)+W(t,p)$
- La séquence $\sigma=t_1t_2...t_n$ est franchie, noté $M(\sigma>M_n$
  - Ssi $\exists M_1,...,M_n$ tels que $M(t_1>M_1(t_2>M_2(...M_{n-1}(t_n>M_n$

Langage d'un RdP étiqueté

- Si on étiquette les transitions d'un RdP sur un alphabet Σ, on mot produit par le RdP est la suite des étiquettes des transitions d'une séquence franchissable $M(\sigma>M_n$
- Pour simplifier les notations, si on considère que les transitions sont nommées sur des lettres de Σ
- Langage du RdP: $L(N,M_0)=\{\sigma \in T^* \mid M_0(\sigma>\}$
- Langage terminal: $L(N,M_0,\mathbb{M})=\{\sigma \in T^* \mid \exists M \in \mathbb{M}, M_0(\sigma>M\}$
- Marquages accessibles du RdP :
  - $Acc(N,M_0)=\{M \mid \exists\ \sigma \in L(N,M_0),\ M_0(\sigma>M\}$

Expressivité

- Proposer un automate fini dont le langage est $a^n b^n$ pour $n \leq 2$
- Proposer un RdP pour ce langage
- Proposer un automate fini dont le langage est $a^n b^n$ pour n quelconque
- Proposer un RdP pour ce langage

Etude d'un RdP

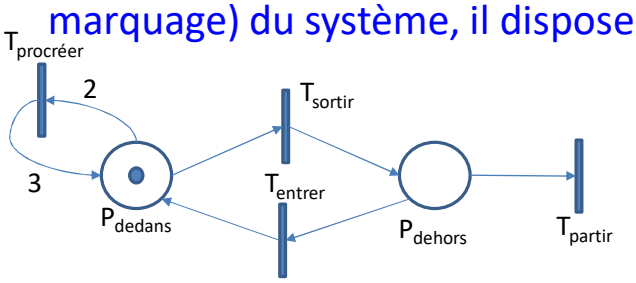
- Propriétés de vivacité
  - Pseudo-vivant : aucun état de blocage
  - Vivant : tout état (et donc tout état souhaitable) est toujours atteignable
- Sûreté
  - Rien de mauvais ne pourra se produire
    - ✓ Place observateur, contenant des jetons lorsque quelque chose à éviter se produit

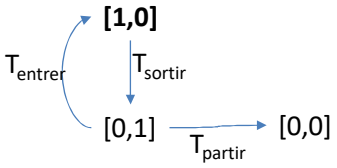
Formalisation propriétés

- (N,M_0) admet une **séquence infinie**
  - $\Leftrightarrow \exists \sigma_\infty \in T^\infty / \forall \sigma$ préfixe fini de σ_∞, $M(\sigma>$
- (N,M_0) est **pseudo-vivant**
  - $\Leftrightarrow \forall M \in Acc(N,M_0), \exists t \in T, M(t>$
- (N,M_0) est **quasi-vivant**
  - $\Leftrightarrow \forall t \in T, \exists M \in Acc(N,M_0)/M(t>$
  - $\Leftrightarrow \forall t \in T, \exists \sigma \in L(N,M_0)/M_0(\sigma t>$
- (N,M_0) est **vivant**
  - $\Leftrightarrow \forall M \in Acc(N,M_0)$, (N,M) quasi-vivant
  - $\Rightarrow (N,M_0)$ quasi-vivant et pseudo-vivant
  - $\Leftarrow (N,M_0)$ quasi-vivant et M_0 état d'accueil

Vivacité

- Dans les deux GM précédents, le RdP est **vivant**. Quel que soit l'état (le marquage) du système, il dispose encore de toutes ses fonctionnalités.



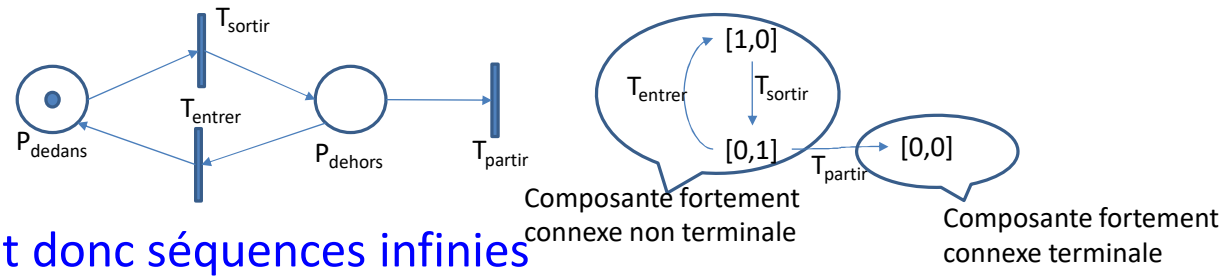


- Ici, il existe un **blocage**, le marquage [0,0] est un marquage **mort** (état puits).
- A partir du marquage [1,0] le réseau n'est pas **quasi-vivant**, car $T_{procréer}$ n'est jamais franchissable
- A partir du marquage [2,0] le réseau est **quasi-vivant**, mais pas **pseudo-vivant**

Propriétés relatives à l'état

- M **état d'accueil** de (N,M_0)
  - $\Leftrightarrow \forall M' \in Acc(N,M_0), \exists \sigma \in T^*/M'(\sigma>M$
- (N,M_0) est **borné**
  - $\Leftrightarrow Acc(N,M_0)$ est fini
  - $\Leftrightarrow \forall p \in P, \exists k_p \in \mathbb{N}/M(p) \leq k_p$
  - $\Leftrightarrow \exists k \in \mathbb{N}/\forall p \in P, M(p) \leq k$
- Monotonie: $M(t>M' \wedge M_1 \geq M \Rightarrow M_1(t>M'_1 \wedge M'_1 \geq M'$
- $M(\sigma>M' \wedge M' \geq M \Rightarrow M(\sigma^n>$
  - Si $M'>M$ réseau non borné

Analyse par graphe des marquages

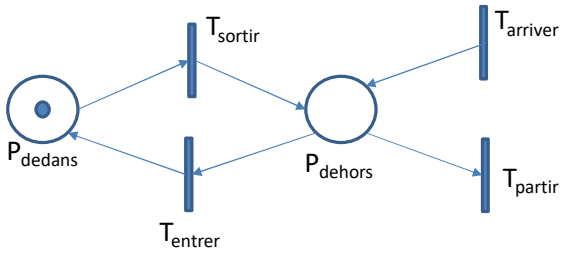


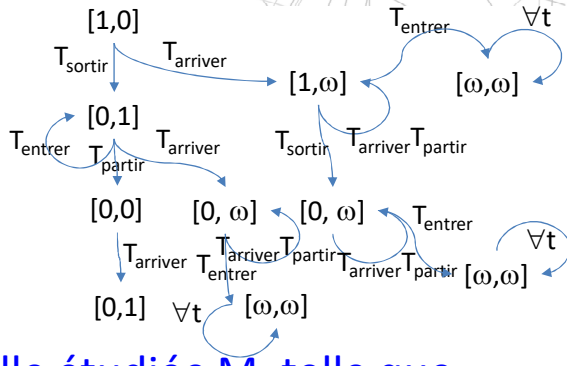
- 1 circuit donc séquences infinies
- Sommet sans successeur donc non pseudo-vivant (donc non vivant, pas d'état d'accueil)
- Toutes les transitions apparaissent donc quasi-vivant
- Borné par 1

Analyse par graphe des marquages

- Cas (N,M_0) borné
  - (N,M_0) admet une séquence infinie $\Leftrightarrow$ GM possède un circuit
  - (N,M_0) vivant $\Leftrightarrow$ $\forall$composante fortement connexe terminale contient un arc étiqueté par toute transition de T
  - (N,M_0) état d'accueil $\Leftrightarrow$ $\exists$ une unique composante fortement connexe (terminale)
- (N,M_0) quelconque
  - (N,M_0) pseudo-vivant $\Leftrightarrow$ tout sommet admet un successeur
  - (N,M_0) quasi-vivant $\Leftrightarrow$ le GM contient un arc étiqueté par toute transition de T

Arbre de couverture

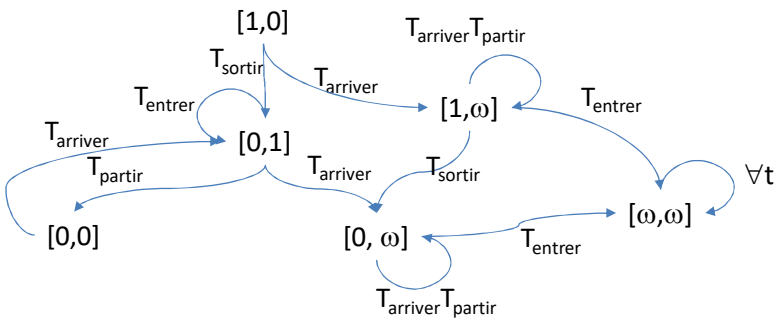




- En largeur ou profondeur, à partir de la feuille étudiée M, telle que $M_0(\sigma>M$. Si M existe dans son préfixe (i.e. $\sigma=\sigma_1\sigma_2$ / $M_0(\sigma_1>M(\sigma_2>M)$, ne pas le traiter. Sinon
  - $\forall t/M(t>M'$, si il existe sur le chemin $M_0(\sigma>M'$ un marquage $M/M'>M$, remplacer chaque composante supérieure par ω.
  - Règles algébriques: $\forall n\in\mathbb{N}$, $\omega+n=\omega-n=\omega$, $\omega>n$, $\omega\leq\omega$

Graphe de couverture

- Fusionner les états identiques



Complexité analyse

- Borné par k : PSPACE-complet (les plus compliqués des problèmes requérant une taille polynomiale pour être résolus, suspecté d'être plus complexe que NP-complet)
- Accessibilité
  - PSPACE-complet si les transitions du RdP ont le même nombre de places en pré qu'en post
  - PSPACE-complet pour les RdP saufs (bornés par 1)
  - NP-complet si pas de cycle
  - Double exponentiel si au plus 5 places
  - EXPSPACE-complet pour les RdP symétriques (∀transition, il existe une transition « l'annulant »)
  - Etc.

Complexité analyse

- Vivacité
  - PSPACE-complet pour RdP saufs
- Sans blocage
  - PSPACE-complet pour RdP saufs s'ils sont à chemin simple (∃ une seule séquence infinie de transitions)
- Etat d'accueil : décidable

Modélisation par RdP

- Proposer un modèle RdP pour le programme parallèle suivant

<u>Faire toujours</u>
Attendre 2 vis
Assembler et créer élément

<u>Faire toujours</u>
Produire 1 vis

- Proposer un modèle RdP pour le programme parallèle suivant

<u>Faire toujours</u>
Attendre 2 vis et 2 boulons
Assembler et créer élément

<u>Faire toujours</u>
Produire 1 vis

<u>Faire toujours</u>
Produire 3 boulons

- Proposer un modèle RdP pour le programme suivant

Si i=0 alors A
Sinon B

Valeur i

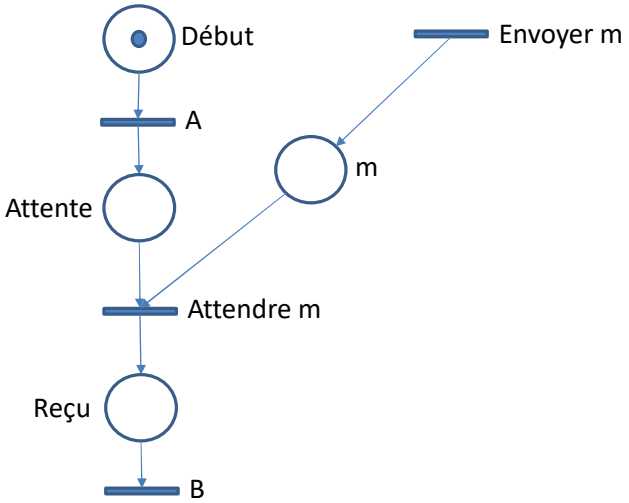
Détection de blocage

Processus Marius
Faire toujours
    Jouer une carte
    Ou
    Prendre le seau de glace
    Prendre boisson anisée et se servir

Processus Panis
Faire toujours
    Jouer une carte
    Ou
    Prendre boisson anisée
    Prendre le seau de glace et se servir

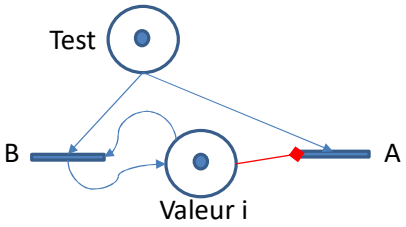
Sémantique d'un RdP

- Modéliser le programme suivant avec RdP
  - Faire action A
  - Attendre message m
  - Faire action B



Extension : arc inhibiteur

- Arc disant qu'une transition est franchissable si la place ne contient pas de jeton
- Augmente la puissance du RdP à celle d'une machine de Turing



Extension vitesse maximale

- Le franchissement des transitions devient déterministe, tout se passe comme si chaque transition avait une durée 1 et se devait d'être tirée en durée 1
- Augmente la puissance du RdP à celle d'une machine de Turing

Autres extensions

- RdP coloré
  - Couleurs de jetons, couleurs de valuations d'arcs
  - Permet de fusionner des places/transitions
- RdP à file
  - Les places contiennent des files FIFO de jetons, les arcs sont valués par des FIFO aussi
  - Modélisation d'échanges avec données ordonnées

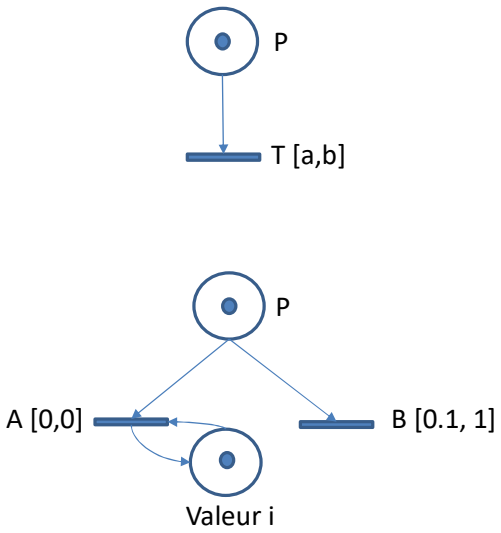
P. Merlin & D. Farber 1976

RÉSEAUX DE PETRI TEMPORELS
TIME PETRI NETS

Familles de RdP à temps

- Deux familles
  - RdP temporisés (timed Petri nets) : chaque transition possède une durée fixe – variantes où la place possède une durée de rétention du jeton
  - RdP temporels (time Petri nets) : les transitions possèdent un intervalle de temps de franchissement
    - ✓Très puissants pour modéliser l'incertitude du temps, par exemple durée d'exécution de tâche, durée de transmission, etc.

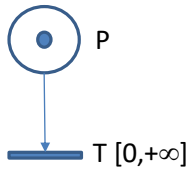
Principe



- A partir de l'instant t auquel la transition est franchissable au sens classique, elle devra être franchie entre les instants t+a et t+b inclus.
- Comme A est forcée de tirer dès qu'elle est franchissable mais que B doit attendre 0.1, on a ici un test à 0.

Formalisation

- Réseau de Petri : $N=(P,T,W,\alpha,\beta,M_0)$ où
  - P ensemble fini de places, $M_0 \in \mathbb{N}^{|P|}$ marquage initial
  - T ensemble fini de transitions $(P \cap T)=\varnothing$
  - W: $P \times T \cup T \times P \rightarrow \mathbb{N}$ fonction de valuation
    - ✓Pré $W_{|P \times T}$ fonction de précondition
    - ✓Post $W_{|T \times P}$ fonction de postcondition
  - Date de tir statique au plus tôt $\alpha: T \rightarrow \mathbb{Q}^+$
  - Date de tir statique au plus tard $\beta: T \rightarrow \mathbb{Q}^+ \cup \{+\infty\}$
    - ✓Intervalle de tir statique de t: $[\alpha(t), \beta(t)]$ avec $\alpha(t) \leq \beta(t)$



Sémantique RdP classique

Sémantique

- Transition $t \in T$ valide si $M \geq Pré(t)$
- Valide(M) est l'ensemble des transitions valides pour M
- Une transition t_k est **nouvellement** valide après le franchissement de t_i si ($t_k \notin Valide(M-Pre(t_i))$ et $t_k \in Valide(M-Pre(t_i)+Post(t_i))$) ou ($t_i=t_k$).
- Initialement, toute transition valide est nouvellement valide

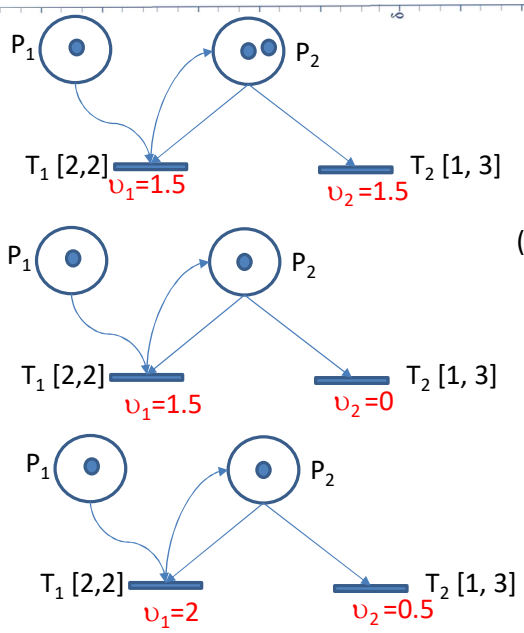
Sémantique

- L'état d'un RdP temporel N est définie par (M,Y)
  - $M \in \mathbb{N}^{|P|}$ marquage du RdP
  - $Y \in \mathbb{R}^{+|T|}$ horloges locales de transitions
- Une transition **continue** fait avancer les horloges d'un délai $d \in \mathbb{R}_{\geq 0}$ dans la limite de leur date de tir statique au plus tard β
  - $(M, Y) \xrightarrow{d} (M, Y+d)$ avec $\forall t \in$ Valide(M), $Y(t)+d \leq \beta(t)$
- Une transition **discrète** franchit une transition franchissable t (transition valide dont l'horloge locale a atteint sa date de tir statique au plus tôt) et réinitialise les horloges locales des transitions nouvellement valides
  - $(M, Y) \xrightarrow{t} (M', Y')$ avec $t \in$ Valide(M), M'=M-Pre(t)+Post(t), $Y(t) \geq \alpha(t)$, $\forall t'$ nouvellement valide, $Y'(t')=0$, pour les autres $Y'(t')=Y(t')$

Sématique

- Une transition est une séquence d'une transition continue suivie d'une transition discrète
  - $(M, Y) \xrightarrow{d} (M, Y+d) \xrightarrow{t} (M', Y')$
- Elle peut s'écrire
  - $(M, Y) \xrightarrow{d,t} (M', Y')$
- Une séquence de transitions est la séquence $(d_1,t_1)(d_2,t_2)$… de transitions $(M, Y) \xrightarrow{d1,t1} (M_1, Y_1) \xrightarrow{d2,t2} (M_2, Y_2)$…
- La séquence non temporisée est t_1t_2…

Exemple



(M,Y)=([1,2],[0,0])

Transition continue de d=1.5

(M,Y+1.5)=([1,2],[1.5,1.5])

Transition discrète de T_2: ([1,2],[1.5,1.5])(T2>([1,1],[1.5,0])

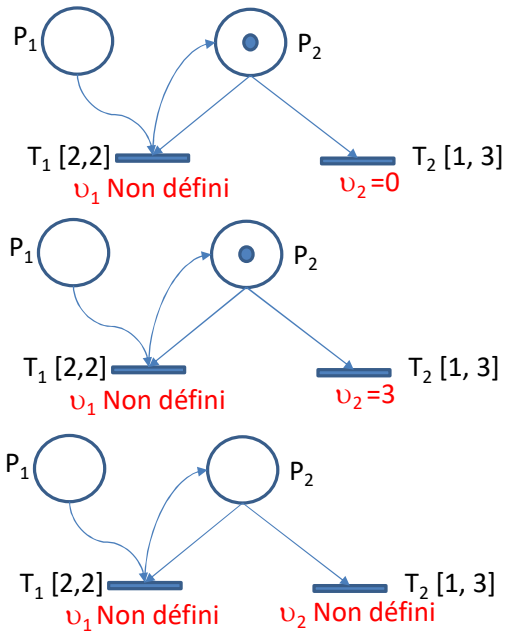
T_2 nouvellement franchissable

(M,Y')=([1,1],[1.5,0])

Transition continue de d=0.5

(M,Y'+0.5)=([1,1],[2,0.5])

Exemple/suite



Transition discrète de T_1: ([1,1],[2,0.5])(T_1>([0,1],[∅,0])

T_2 nouvellement franchissable

(M,Y'')=([0,1],[∅,0])

Il est inutile de tenir le compte

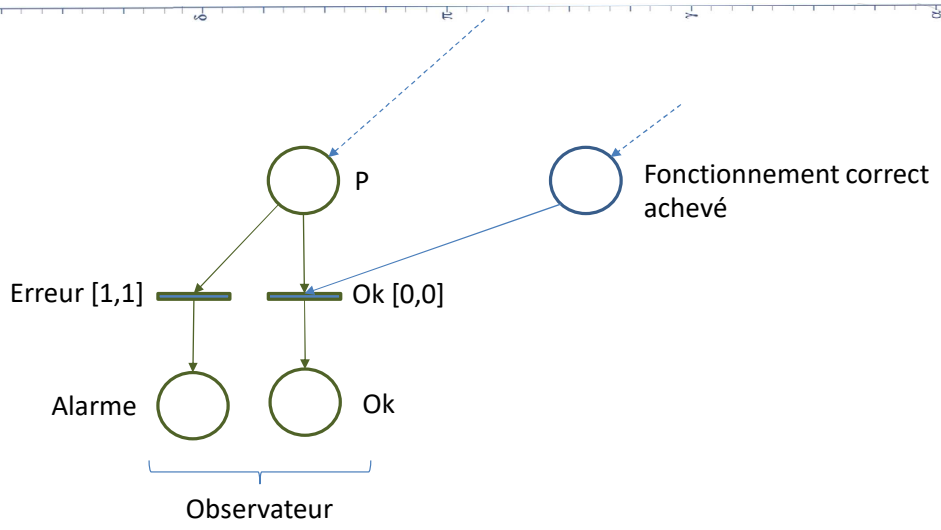
Transition continue de d=3

(M,Y''+3)=([0,1],[∅,3])

Transition discrète de T_2: ([0,1],[∅,3])(T2>([0,0],[∅, ∅])

(M,Y''')=([0,0],[∅,∅])

Modélisation de chiens de garde



Propriétés

- Marquage atteignable
- Pseudo-vivacité, quasi-vivacité, vivacité
- Logique temporelle: plusieurs logiques temporelles peuvent être étudiées sur un RdP temporel
  - LTL, CTL, CTL*

Linear Tree Logic LTL

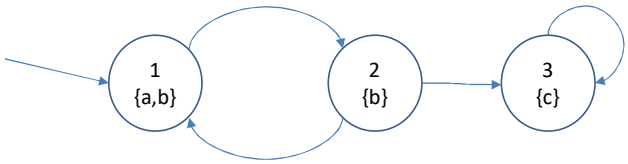
- Dans notre contexte, si l'on considère une séquence non temporisée de transitions $w=t_1t_2$… on peut vouloir vérifier des propriétés LTL (par exemple sur les marquages atteints sur une séquence possible):
  - neXt(f) noté Xf : f est vraie à la prochaine étape
  - Globally(f) noté Gf: f est vraie dans tous les états à partir de maintenant
  - Finally(f) noté Ff : f est vraie dans un état futur
  - f Until g noté fUg : f vraie jusqu'à ce que g soit vraie (g doit être vraie à un certain point)
  - f Weakuntil g noté fWg :comme U sauf que f peut être toujours vraie (auquel cas g n'est pas forcément vraie à un certain point)
    - ✓ $fWg \equiv (fUg) \vee Gf$

Structure de Kripke

$M = (S, I, L, \rightarrow)$

Système de transitions de Kripke, S ensemble d'états, $I \subseteq S$ états initiaux, $\rightarrow \subseteq S \times S$ transitions tel que tout état possède au moins un successeur, fonction d'interprétation $L: S \rightarrow 2^{PA}$ où PA ensemble de propositions atomiques

Deadlock représenté par transition en boucle sur le même état



$M = (S = \{1,2,3\}, I = \{1\}, L, \rightarrow)$
L(1)={a,b}, etc.
PA={a,b,c}

- b est-il vrai sur tout chemin à partir de 1 ?
  - $(M,1) \not\models AGb$ cependant $(M,1) \models AFb$
- b est-il vrai sur tout chemin jusqu'à ce que c soit vrai à partir de 1 ?
  - $(M,1) \models A(bWc)$
- Existe-t-il un chemin à partir de 3 tel que b soit vrai ?
  - $(M,3) \not\models EFb$

Computation Tree Logic (CTL et CTL*)

- S'applique sur un arbre de chemins possibles dans une structure de Kripke (le graphe des marquages d'un RdP ou le graphe de couverture peut être vu comme une telle structure, l'interprétation étant représentée par le marquage)
- Exprime des propriétés LTL avec quantificateur de chemin (∀ A All, ∃ E Exists)
  - Af : f vraie sur tout chemin (à partir de l'état courant)
  - Ef : f vraie sur au moins un chemin (à partir de l'état courant)
  - X, G, F, U, W : comme dans LTL
- Une structure de Kripke n'étant pas temporisée, ne considère pas le temps absolu, mais la causalité

Sémantique CTL*

- Formule d'état : $M = (S, I, L, \rightarrow)$ structure de Kripke, $s \in S$, Φ formule d'état. $(M,s) \models \Phi$ se lit l'état s satisfait la formule Φ
- $(M,s) \models \top$ $\quad (M,s) \not\models \bot$
- $(M,s) \models p \Leftrightarrow p \in L(s)$
- $(M,s) \models \neg\Phi \Leftrightarrow (M,s) \not\models \Phi$
- $(M,s) \models \Phi_1 \wedge \Phi_2 \Leftrightarrow ((M,s) \models \Phi_1) \wedge ((M,s) \models \Phi_2)$
- $(M,s) \models \Phi_1 \vee \Phi_2 \Leftrightarrow ((M,s) \models \Phi_1) \vee ((M,s) \models \Phi_2)$
- $(M,s) \models \Phi_1 \Rightarrow \Phi_2 \Leftrightarrow ((M,s) \not\models \Phi_1) \vee ((M,s) \models \Phi_2)$
- $(M,s) \models A\phi \Leftrightarrow$ (∀chemin σ partant de s, $\sigma \models \phi$)
  - ϕ formule de chemin, $\sigma = s_1 s_2 s_3 \ldots$ tel que $s_1 = s$, $s_1 \rightarrow s_2$, $s_2 \rightarrow s_3$, etc.
- $(M,s) \models E\phi \Leftrightarrow$ (∃ chemin σ partant de s, $\sigma \models \phi$)

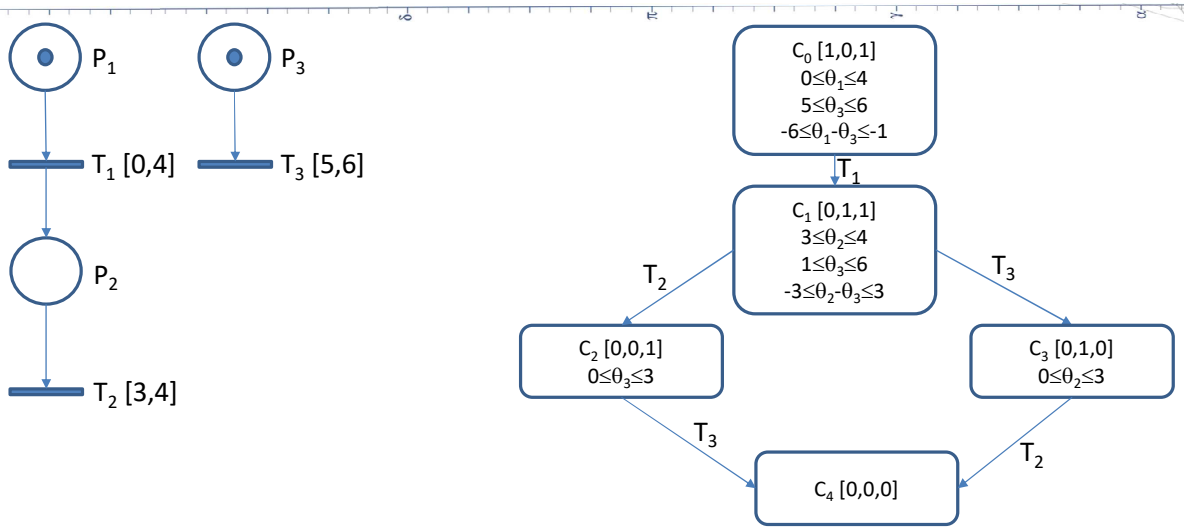
CTL* formules de chemin

- Soit un chemin $\sigma = s_0 \rightarrow s_1 \rightarrow \ldots$. On note le suffixe $\sigma(i)$ du chemin le chemin $s_i \rightarrow s_{i+1} \rightarrow \ldots$
- $\sigma \models X\phi \Leftrightarrow \sigma(1) \models \phi$
- $\sigma \models F\phi \Leftrightarrow (\exists i \geq 0 / \sigma(i) \models \phi)$
- $\sigma \models G\phi \Leftrightarrow (\forall i \geq 0 / \sigma(i) \models \phi)$
- $\sigma \models \phi_1 U \phi_2 \Leftrightarrow (\exists i \geq 0 / \sigma(i) \models \phi_2 \wedge \forall 0 \leq j < i, \sigma(j) \models \phi_1)$

Etude de comportement

- Le TPN s'exécute en temps continu, en théorie il y a donc une infinité d'états possibles
- Il faut regrouper des états « équivalents » de façon à obtenir un graphe fini si le marquage est fini
  - Graphe de classes
  - Graphe de régions géométriques
  - Graphe de classes fort
  - Graphe basé zones
  - Graphe de classes atomique
- Cependant, en fonction du regroupement un certain nombre de propriétés LTL ou CTL* ou temporisées sont perdues

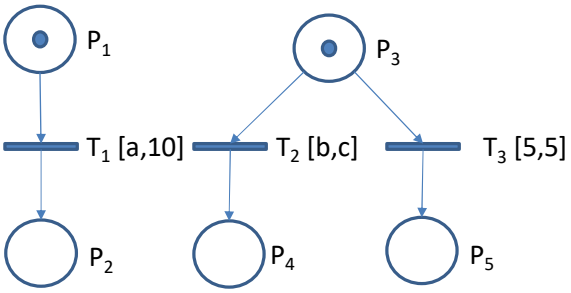
Graphe de classes



- Remarque: si θ_1 vaut 0, en réalité seule la séquence T_2,T_3 est permise, alors que si θ_1 vaut 4, seule T_3,T_2 est permise. Le graphe des classes ne le montre pas. Il conserve cependant les propriétés LTL et CTL* qui ne se basent pas sur du temps.

TPN paramétrique

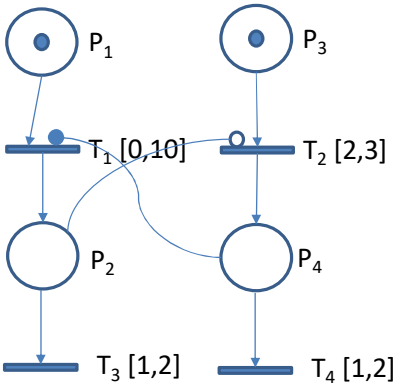
- On ajoute à la définition du TPN
  - Par={λ_1, λ_2,…, λ_l} ensemble de paramètres
  - $D_{Par} \subseteq \mathbb{Q}^{+Par}$ polyèdre convexe contraignant les paramètres



Par={a,b,c}
0≤a≤10
0≤b≤c

Arcs inhibiteurs et stopwatchs

- Inhibiteur logique : transition non valide si place amont marquée ●
- Inhibiteur stopwatch : transition non valide si place amont marquée, mais horloge locale conservée ○



Si T_1 franchie au bout d'une unité de temps, P_2 est marquée, et l'horloge locale de T_2 vaut 1, tout se passe donc comme si l'intervalle de franchissabilité de T_2 était [1,2]. Cette horloge est « gelée » par la stopwatch.
Si T_3 est franchie n'importe quand dans [1,2], à partir de cet instant, l'horloge de T_2 est « dégelée » et T_2 pourra être franchie entre 1 et 2 unités de temps plus tard. T_2 n'est pas nouvellement valide (horloge locale non réinitialisée)
Si T_2 est franchie en premier, T_1 n'est pas valide tant que P_4 est marquée. Au bout d'une à deux unités de temps, T_4 est franchie, T_1 est nouvellement valide.
On peut mettre des poids sur les inhibiteurs et stopwatchs, ils jouent alors leur rôle inhibiteur si la place amont est marquée par des jetons dont le nombre est au moins le poids.

Analysibilité

- De façon générale, l'accessibilité d'un TPN avec paramètres et inhibiteurs n'est pas décidable
- Si il n'existe que des paramètres en tant que borne inférieure ou supérieure d'intervalle de tir statique, et qu'aucun n'est à la fois borne inférieure et supérieure de deux intervalles différents, l'accessibilité est décidable.
- Construction d'un graphe de classe paramétrique sur un principe similaire au graphe de classes

TPN-TCTL

- Generalized Mutual Exclusion Constraint GMEC
- GMEC: GMEC and GMEC | GMEC or GMEC | GMEC => GMEC | not GMEC | $\sum_{i=1}^{m} a_i M(p_i)$ COMP c
  - $a_i \in \mathbb{Z}$, M marquage, p_i place du RdP
  - COMP$\in\{<,\leq,=,\geq,>\}$, c$\in\mathbb{N}$
- TPN-TCTL s'exprime sur une structure de Kripke (graphe des zones paramétriques) dans laquelle la fonction d'interprétation est une combinaison linéaire exprimée à base des marquages

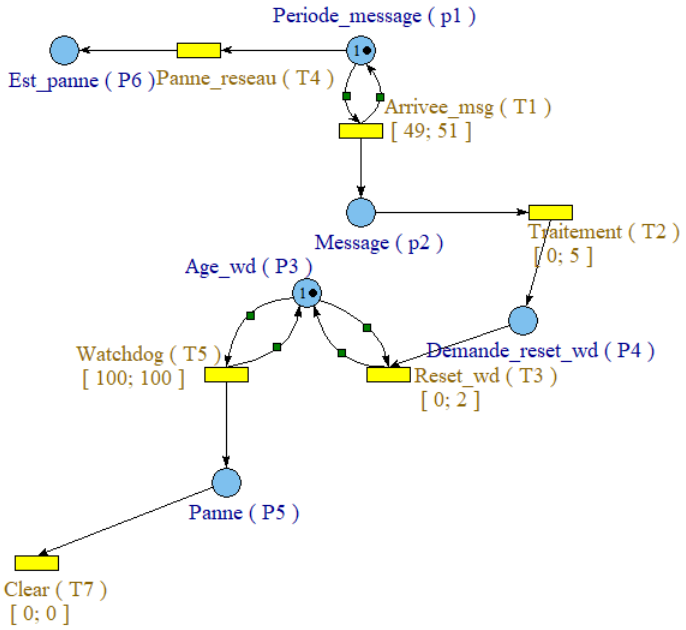
ParamTPN-PTCTL

- EφU[a,b]ψ
  - φ,ψ GMEC
  - a,b rationnels ou paramètres
  - Il existe un chemin possible dans lequel pour une date r∈[a,b] tel que ψ vraie et que pour toute date antérieure, φ vraie
- AφU[a,b]ψ
  - Pour tout chemin possible …
- EF[a,b]φ : il existe un chemin sur lequel on trouve une date dans [a,b] telle que φ vraie
- AF, EG, AG
- φ-->[a,b]ψ équivalent à AG(φ=> AF[a,b]ψ)

Exemple

- Un message est (normalement) reçu toutes les 50 ms, mais il y a une incertitude de 2% sur les horloges
- Il est traité en un temps maximal de 5 ms par le système, ce qui réinitialise le watchdog
- Un watchdog est déclenché lorsqu'il n'a pas été réinitialisé pendant 100 ms. Le traitement du watchdog prend au pire 2 ms.
- Quel est le delai maximal entre l'occurrence d'une panne et la fin du watchdog ?

Exemple dans Romeo



(P6==1)-->[0,a](P5==1)

a in [107, inf[

Pour garantir que P5 est marquée après que P6 soit marquée, il faut un intervalle d'au moins 107 u.t.

Conclusion

- Outil puissant de modélisation et d'analyse
- Plus le modèle est puissant, plus il est complexe à analyser
- Certaines analyses sont indécidables dans le cas général
- Le RdP temporel paramétrique à stopwatches peut être utilisé pour analyser finement les comportements possibles d'un système de tâches
  - Cependant, complexité élevée
  - Passage à l'échelle non assuré

TD Modélisation et analyse par Réseaux de Petri

Réseaux de Petri classiques

Exercice 1 Dîner des philosophes

Nous considérons 4 philosophes ayant un fonctionnement similaire : ils pensent ; lorsqu'ils ont faim ils commencent par prendre la baguette à leur gauche ; puis la baguette à leur droite ; puis ils mangent, déposent les baguettes et se remettent à penser. La Figure 1 représente un extrait du RdP à construire avec Roméo pour 4 philosophes et 4 baguettes. En supposant que la position des places baguette$_i$ est représentative des positions des baguettes sur la table, et que la position des transition modélisant le philosophe est aussi représentative de la position du philosophe à la table, nous voyons que le philosophe$_1$ qui fait face à la table prend la baguette à sa gauche avant de prendre la droite.

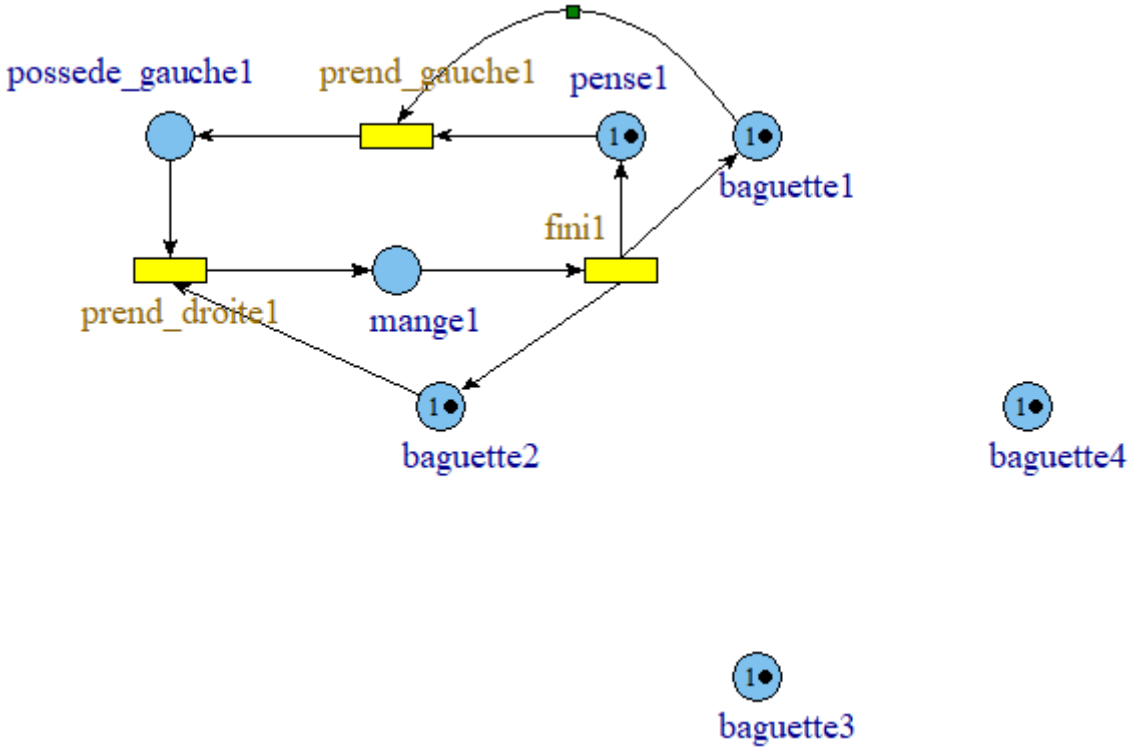


Figure 1 : extrait d'un RdP modélisant le dîner des philosophes à 4 philosophes et 4 baguettes

a) Compléter ce RdP, simuler quelques comportements.
b) Vérifier que le RdP est sauf (1-borné).
c) Tester s'il existe un deadlock.
d) Modifier l'ordre d'accès aux ressources afin qu'il ne soit possible de prendre les baguettes que dans l'ordre croissant de leur numéro. Vérifier à l'aide du *model checker* que le RdP est dorénavant sans deadlock.

Exercice 2 Producteur-consommateur

Producteur consommateur avec buffer en exclusion mutuelle

Lorsqu'il se présente, un producteur est prêt à produire une donnée dans un *buffer*. Déposer la donnée dans un buffer n'étant pas une action atomique, le *buffer* est protégé par un *mutex*, que le producteur va prendre avant de produire dans le *buffer*, et vendre (noter qu'en général dans ce cas, la production et la vente du mutex peuvent se représenter comme une action instantanée, puisque le producteur détient l'accès exclusif au *buffer*). Le consommateur fonctionne de façon duale, et, en assurant l'exclusion mutuelle, va consommer les données du *buffer*.

a) Modéliser ce comportement à l'aide d'un RdP.
b) Simuler le système et vérifier manuellement à l'aide de comportements que votre modèle fonctionne de façon attendue (par exemple respect de l'exclusion mutuelle lors de l'accès au *buffer*).

Attention, *checker* une propriété autre que la bornitude sur un RdP non borné plante en général le model checker de Romeo. D'une façon générale, il faut donc s'assurer de la bornitude d'un RdP avant de lancer une autre vérification de propriété. En cas de plantage du *model checker*, il faut tuer le processus romeo-cli. Sous windows, ce processus n'est pas vu par l'explorateur de processus, et un logiciel type ProcessExplorer sera alors utile pour tuer le processus sans avoir à redémarrer l'ordinateur. Romeo devra être relancé afin que les fichiers ouverts soient fermés et de nouveau accessibles pour le lancement d'une nouvelle analyse.

c) En observant le RdP (mais sans lancer l'analyse automatique) dire si le RdP est borné.

Version bornée du système

d) En s'inspirant de l'implémentation avec deux sémaphores à compte du paradigme du Producteur-Consommateur vu en cours de systèmes d'exploitation, proposer une modification du RdP telle que le *buffer* soit borné par 5.
e) Vérifier à l'aide d'une formule TCTL que le RdP est borné par 5.
f) S'assurer qu'il est possible que le *buffer* contienne 5 messages.

Exercice 3 Lecteur-écrivain

Nous voulons modéliser un système de salle d'examen contenant 5 places, dans laquelle des étudiants entrent et utilisent un poste de travail. Ils planchent individuellement sur le poste de travail, puis, lorsqu'ils ont terminé, vont poser leur devoir dans un *buffer*. Le *buffer* peut contenir jusqu'à 4 documents. Parallèlement à cela, un processus de correction automatique peut se lancer à tout moment : il va interdire l'accès au *buffer* lorsqu'il se lance, va corriger les documents un à un jusqu'à vider le *buffer*, puis autoriser à nouveau l'accès au buffer lorsqu'il a fini.

a) Modéliser ce comportement à l'aide d'un RdP et simuler manuellement quelques comportements.
b) Vérifier que le RdP est borné par 5.

Remarque : dans Romeo, il est pratique de nommer les places et transitions par un nom en rapport avec leur sémantique, mais cela est fort moins pratique pour faire référence aux places ainsi nommées lors de l'écriture d'une GMEC. En fait, les éléments nommés ont un label et un identificateur, l'identificateur étant le nom à utiliser dans une formule d'analyse. Il est donc conseillé de décocher l'option « identifier <- label » et de donner un label simple (type p1, p2, etc.) aux places utilisées dans des GMEC.

c) Vérifier que le *buffer* est borné par 4.

d) Vérifier que le *buffer* peut contenir jusqu'à 4 documents.
e) Vérifier que lorsqu'il y a correction, aucun étudiant n'a accès au *buffer* et vice-versa.

Réseaux de Petri temporels

Dans Romeo, cliquer sur le bouton Control Panel, et choisir l'extension « Stopwatch T-TPN ». Dans la partie « Parameters », choisir « Real parameters ».

Exercice 4 Dimensionnement de buffer

Nous considérons un *buffer* de sortie de switch avionique ARINC664p7 (AFDX™) que nous allons dimensionner en fonction des flux le traversant. Nous supposons que chaque trame de notre application traversant le réseau fait au plus 1kO. Les messages considérés sont sporadiques en sortie du *end-system* les émettant, avec un délai-minimal entre deux envois successifs appelé BAG (*Bandwidth Allocation Gap*). Cependant, dans une première étude nous allons les considérer périodiques.

Nous considérons quatre VL (*virtual links*) : un VL représente un chemin suivi par un message entre un nœud d'origine et un ou plusieurs nœuds de destination, à travers des switchs. Dans notre système, il n'y a qu'un switch, relié par un port bidirectionnel et des paires de câbles Ethernet dédiés 100 Mbps à chaque End/System. Nous considérons les VL 1 à 3, qui sont ceux qui empruntent en particulier la sortie du switch vers l'E/S4.

VL1 : E/S1 -> Switch -> E/S4

VL2 : E/S2 -> Switch -> E/S4

VL3 : E/S3 -> Switch -> E/S4

Le switch garantit qu'un message arrivant sur un port sera déposé à destination en 13µs au plus.

Id message	VL emprunté	BAG (ms)
1.1	1	1
1.2	1	2
1.3	1	4
2.1	2	4
3.1	3	8
4.1	3	1

a) Quelle est la durée d'émission d'une trame d'1kO sur un bus à 100 Mbps ?
b) Sachant que Romeo permet de n'exprimer les intervalles de tir statique qu'en nombres entiers, quelle unité choisir ?
c) Est-il nécessaire ici de représenter dans un modèle visant à dimensionner le *buffer* la vitesse de traversée d'un brin Ethernet ?
d) Modéliser le système par RdP temporel, en supposant que les messages sont émis périodiquement, sachant que nous nous intéressons à la taille nécessaire en sortie de switch.
e) Rechercher par dichotomie le marquage maximal de la place de sortie du switch, en déduire, en kO, la mémoire à prévoir dans ce buffer.
f) Les messages sont considérés sporadiques, modifier le RdP et tester la même propriété. Expliquer pourquoi le *model checker* ne répond pas.

Exercice 5 Ordonnancement à priorités fixes aux tâches de tâches indépendantes

Soit le système de trois tâches périodiques indépendantes à échéance sur requête données sous la forme $<C_i,T_i>$, avec τ_1 et τ_2 périodiques et τ_3 sporadique. $\tau_1<2,10>$, $\tau_3<3,10>$, $\tau_2<2,12>$. Nous choisirons

l'affectation de priorités RM qui est optimale dans ce contexte, et, afin de nous préparer à la suite et aux modifications de l'exercice qui arriveront par la suite, nous choisissons arbitrairement d'éviter les ex-aequo en affectant une priorité supérieure à τ_1.

a) Modéliser le système par un RdP temporel. Comme d'habitude, on s'assurera par simulation que le modèle semble se comporter correctement, et on vérifiera qu'il est borné avant de lancer d'autres analyses.
b) Calculer le pire temps de réponse de chaque tâche à l'aide d'une formule ParamTPN-PTCTL

Jusqu'ici pas vraiment d'intérêt à ce modèle, bien plus complexe à analyser qu'une RTA qui est dans ce cas exacte et nous donnerait le même résultat. Cependant, cela change avec l'adjonction de dépendances, telles que les ressources critiques ou les contraintes de précédence. Modifions notre modèle afin de représenter le système donné en exercice 4 du TD 5, et donné sur la Figure 2.

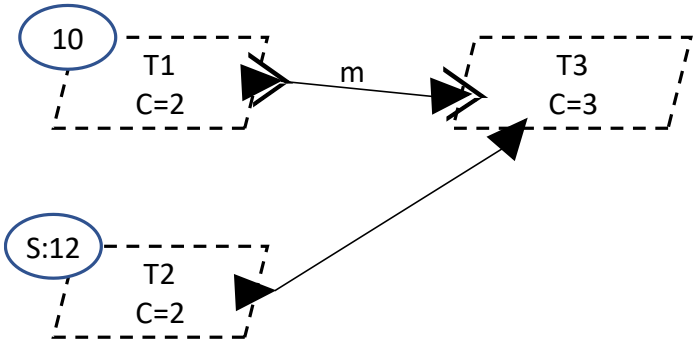


Figure 2 : Système de tâches avec exclusion mutuelle et précédence

Nous supposons que l'écriture (resp. lecture) dans un tableau noir s'opère en au plus 0,5 unité de temps en fin (resp. début) de tâche, alors que l'envoi (resp. réception) de message s'opère en au plus 0,6 unités de temps en fin (resp. début) de tâche. Nous supposons que le système est implémenté sur POSIX, avec l'utilisation du protocole de gestion de ressources à priorités plafonds (PCP) dans sa version immédiate.

c) Modifier le RdP précédent pour représenter les comportements possibles du système. Noter qu'il va nous falloir représenter des dixièmes d'unités de temps, Romeo n'autorisant que des valeurs entières pour les intervalles de tir statiques.
d) Calculer le temps de réponse de chaque tâche, ainsi que le temps de réponse de bout-en-bout du réveil de T1 à la terminaison de T3.

Exercice 6 Communication sans fil

Un drone doit utiliser une liaison radio pour envoyer des messages périodiques vers une station sol. La durée des messages dépend de leur taille, qui est variable en fonction des envois, et du débit de la liaison. Nous supposons l'existence de trois messages périodiques : m_1, de durée maximale 3 millisecondes (ms) et de période 10 ms ; m_2 de durée maximale 6 ms et de période 20 ms, et m_3 de durée maximale 15 ms et de période 30 ms.

a) Quelle est l'unité de temps à employer dans un réseau de Petri temporel ne manipulant que des entiers ? Justifier votre réponse.
b) Dans une première approche, nous supposons que chaque message est envoyé sur sa propre liaison radio, de façon à avoir un modèle simple à partir duquel nous construirons le reste. Proposer un réseau de Petri temporel modélisant l'envoi périodique des messages m_1 à m_3.

c) Afin d'éviter aux messages d'arriver simultanément, on laisse m_1 arriver dès le début, mais m_2 est décalé initialement de 3 ms avant d'être émis périodiquement aux dates 3 ms, 23 ms, 43 ms, etc. et m_3 est décalé initialement de 6 ms. Compléter le RdP précédent de façon à obtenir ce comportement.

d) En fait, il n'y a qu'une seule liaison radio, et les messages seront envoyés de façon non préemptive conservative. Cela signifie que s'il y a au moins un message à envoyer lorsque le canal est libre, un message est envoyé. Cependant nous n'avons pas connaissance de la stratégie employée pour choisir le message si plusieurs messages sont prêts lorsque le canal est libre. Proposer un RdP inspiré du RdP précédent permettant de prendre en compte cette politique. L'idée générale consiste à réserver la liaison dès que possible lorsqu'un message est prêt, et ne la libérer qu'à la fin de l'émission du message.

e) Exprimer en logique temporelle de Roméo comment calculer le pire temps de réponse du message m_1.

f) Exprimer en logique temporelle de Roméo comment calculer le pire temps de réponse du message m_3.

Nom :
Prénom :

Nom :
Prénom :

Nom :
Prénom :

Nom :
Prénom :